Evolving Block-Based Convolutional Neural Network for Hyperspectral Image Classification

Zhenyu Lu, *Member, IEEE*, Shaoyang Liang, Qiang Yang, *Member, IEEE*, and Bo Du, *Senior Member, IEEE*

***Abstract*— Deep convolutional neural network (CNN) shows excellent effectiveness on hyperspectral image (HSI) classification. However, the architecture design of CNN requires abundant expert knowledge and experience, which poses great prohibition to its wide application in real-world engineering. To alleviate the issue, this article proposes an evolving block-based CNN (EB-CNN) to search the optimal architecture based on the genetic algorithm (GA) automatically. Specifically, two kinds of basic blocks with totally six different configurations are first designed to construct the search space. Then, a flexible encoding strategy is devised for the GA to allow different chromosomes to evolve with different lengths. In this manner, the width of each layer and the depth of the architecture can be simultaneously optimized. Furthermore, a novel swapping mutation operator is proposed for the GA to speed up the search efficiency and save computing resources. With the abovementioned techniques, the proposed algorithm automatically seeks the optimal CNN architecture for HSI classification, leading to its better usability than hand-crafted CNNs. At last, extensive experiments conducted on five commonly used HSI datasets demonstrate that the proposed EB-CNN achieves highly competitive or even better performance, as compared with the state-of-the-art peer algorithms.**

***Index Terms*— Block-based convolutional neural network (CNN), CNN, evolving CNN, hyperspectral image (HSI) classification, neural architecture search (NAS).**

I. Introduction

HYPERSPECTRAL image (HSI) classification plays a fundamental role in remote sensing. It has been applied in various practical cases, such as forest monitoring [1], geological prospecting [2], water pollution detection [3], and so on. Different from normal images obtained from normal cameras, HSIs are collected by hyperspectral sensors with a wide wavelength range of the electromagnetic spectrum. Therefore, they contain rich spectral and spatial information along with massive connotation information for image recognition and classification [4], [5]. Furthermore, unlike normal image classification, which is a kind of image-level or object-level classification, HSI classification is a kind of pixel-level classification [6]. In particular, HSIs bring many challenges to the existing classification techniques. For example, it often occurs that two objects are with the same spectral signatures or two spectral bands are with the same object features in an HSI [6]–[8]. Besides, the imbalance problem also commonly exists in different types of samples in an HSI [9].

To solve the abovementioned challenges, researchers have been devoted to designing novel classification approaches [10]–[15]. In a broad sense, existing HSI classification methods can be roughly divided into two categories: 1) traditional machine learning-based methods [10]–[12] and 2) deep learning-based methods [13]–[15].

Traditional machine learning-based methods mainly contain two separate steps: feature extraction and classifier training [6], [9]. The first step is mainly to extract discriminative features from HSIs. The commonly used hand-crafted feature extraction methods include random forest [10], local binary patterns (LBPs) [16], and scale-invariant feature transform (SIFT) [17]. Then, based on the extracted features, a classifier is trained [18]–[21]. The commonly used classifiers include sequence binary branch tree [18], sparse representation classification (SRC) [11], multilogistic regression [12], and support vector machine (SVM) [19]–[21].

Although traditional machine learning-based classification methods have achieved promising performance, they heavily rely on feature extraction, which requires a lot of human effort and experience. Since one HSI is a special 3D tensor data with abundant spectral and spatial information, traditional feature extraction methods have limited ability to extract all valuable information [8], [9], [13]. To conquer these limitations, researchers turn to deep learning-based methods due to their powerful feature extraction capabilities [22]–[24]. As a result, a series of handcrafted deep neural network (DNN) (especially convolutional neural network (CNN))-based classification methods have been proposed in recent years [6], [14], [15]. Armed with the powerful ability of DNNs, these methods achieve very promising performance on HSI classification.

However, human expert knowledge and experience are heavily involved in the architecture design of existing CNNs for HSI classification [25]–[27]. For instance, the settings of

Manuscript received December 23, 2021; accepted March 11, 2022. Date of publication March 17, 2022; date of current version April 15, 2022. This work was supported in part by the National Natural Science Foundation of China under Grant U20B2061 and Grant 62006124, in part by the Natural Science Foundation of Jiangsu Province under Project BK20200811, in part by the Natural Science Foundation of the Jiangsu Higher Education Institutions of China under Grant 20KJB520006, and in part by the Startup Foundation for Introducing Talent of the Nanjing University of Information Science and Technology (NUIST). *(Corresponding author: Qiang Yang.)*

Zhenyu Lu and Qiang Yang are with the School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: luzhenyu76@163.com; mmmyq@126.com).

Shaoyang Liang is with the School of Electronics and Information Engineering, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: eminen_sy@163.com).

Bo Du is with the Hubei Key Laboratory of Multimedia and Network Communication Engineering, National Engineering Research Center for Multimedia Software, School of Computer Science, Institute of Artificial Intelligence, Wuhan University, Wuhan 430072, China (e-mail: dubo@whu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TGRS.2022.3160513, provided by the authors.

Digital Object Identifier 10.1109/TGRS.2022.3160513

1558-0644 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

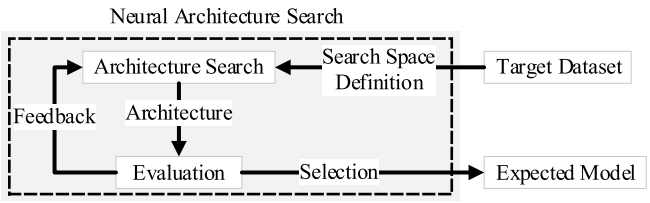


Fig. 1. Overall framework of NAS.

various hyperparameters, such as the convolution kernel size, the pooling type, and the number of layers, are domain-specific and thus need to be carefully set based on prior knowledge on the specific domain. With this constraint, the usability of most existing CNN-based HSI classification methods is limited.

To alleviate the above shortcoming, researchers attempt to design automatic CNNs, leading to a new emerging topic named neural architecture search (NAS). Specifically, the aim of NAS is to take researchers or users out of the trouble of manually designing the CNN architecture by fine-tuning hyperparameters [25], [27]. In particular, the general framework of NAS is shown in Fig. 1. Specifically, given the predefined search space, an optimization algorithm iteratively optimizes the CNN architecture based on the feedback evaluated on a specific dataset. After the given number of iterations, the found optimal architecture along with the trained weight of CNN is output as the final model.

In this direction, evolutionary algorithms (EAs) are frequently adopted as the optimization algorithm [27]–[32] due to their powerful global search capabilities. EAs are a kind of metaheuristic algorithms, which maintain a population to iteratively search the solution space to find the global optima of optimization problems. Broadly speaking, existing EA-based NAS methods can be separated into two categories. The first kind of approaches aims to optimize the component of each layer from the point of the micro view [33]–[35]. These methods usually keep the network architecture fixed and then seek the optimal settings of some cells, such as the kernel size, the type of the loss function, and the type of pooling. Another kind of methods aims to optimize the whole architecture with predefined units from the perspective of the macro view [28]–[30]. Generally, these methods predesign some fixed units, and then look for the best combination and arrangement of these units to form an optimal architecture.

Though various EA-based NAS methods have been proposed and shown promising performance on image classification [29]–[32], on the one hand, most existing NAS methods are particularly designed for normal image classification, not for HSI classification. Since HSI classification is a kind of pixel-level classification, which is different from normal image classification as aforementioned [6], existing NAS methods cannot be directly utilized for HSI classification or obtain poor classification performance without any special design; on the other hand, as mentioned above, most existing EA-based NAS methods generally optimize the architecture either only from the micro view or only from the macro view. Both kinds of methods have limitations to seek the globally optimal architecture.

To alleviate the above limitations, this article proposes an evolving block-based CNN (EB-CNN) for HSI classification via optimizing the architecture simultaneously from the micro view and the macro view, namely, simultaneously seeking the optimal component of each layer and the optimal width and length of the architecture. Specifically, the novelty of the proposed EB-CNN is summarized as follows:

1) An EB-CNN architecture search algorithm is devised to seek the optimal overall architecture by automatically optimizing the components of layers and the width and depth of the architecture simultaneously for HSI classification. Instead of fine-tuning hyperparameters (e.g., the number of layers, the convolution kernel size, the type of pooling, etc.) of CNN empirically with tedious effort as in existing hand-crafted CNN-based HSI classification methods [13]–[15], the proposed EB-CNN adopts the genetic algorithm (GA) as the optimizer in Fig. 1 to seek the optimal architecture automatically as in most existing EA-based NAS studies [28]–[32]. In this way, the dependence on specific knowledge about CNN is declined to a large extent. Therefore, it is expected that the proposed EB-CNN has better usability than existing HSI classification methods.
2) Two novel basic blocks are designed and different configurations of these two blocks are predefined to construct the search space. Instead of predefining the search space with trivial and subtle operations (e.g., node connections and basic cells) as in existing NAS studies [33]–[36], this article predesigns the search space from the macro perspective. In this manner, not only could the reliance on the expert knowledge and experience in the detailed operations of CNN be reduced, but also the search space can be largely reduced, which is beneficial for GA to search the architecture efficiently.
3) A flexible encoding scheme is devised to represent the chromosomes in GA based on the constructed blocks. Instead of encoding all chromosomes with the same length as in existing NAS methods [33]–[35], this article devises a flexible encoding scheme to represent the chromosomes in GA based on the constructed blocks. By this means, different chromosomes have different lengths, and thus the optimal architectures with different depths could be found by EB-CNN for different scenarios. Therefore, the flexibility of the proposed EB-CNN is largely improved. In this way, the reliance on specific prior domain knowledge could be reduced to a large extent.
4) A swapping mutation operator is proposed along with other existing mutation operators to help GA evolve the population efficiently. This operator swaps the blocks in one chromosome and thus can be seen as a local search operator. With this operator, the exploitation ability of GA can be improved, which is beneficial for GA to find the optimal architecture fast, given very limited resources.
5) As a whole, with the above techniques, the proposed EB-CNN cannot only optimize the configurations of the basic blocks (e.g., the type of the block and the number of the blocks), but also optimizes the overall structure

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

of the architecture (e.g., the combination of the blocks, and the width and depth of the architecture). In this way, the adaptability of the proposed EB-CNN to different scenarios could be largely promoted because it can find different suitable CNN architectures for different datasets. To the best of our knowledge, this is the first attempt to completely optimize the overall CNN architecture automatically for HSI classification.

Extensive experiments are conducted on five commonly used HSI benchmark datasets to validate the effectiveness of the proposed method. In addition, deep investigation on the proposed EB-CNN is also conducted to observe the effectiveness of its components.

The rest of this article is organized as follows. Section II reviews closely related works on deep learning-based HSI classification methods. Then, in Section III, the proposed EB-CNN is elucidated in detail. In Section IV, extensive experiments are conducted to verify the effectiveness of the proposed method. Finally, Section V concludes this article and presents some future directions.

II. Related Work

Since this article mainly focuses on developing an automatic DNN for HSI classification, this section only reviews closely related works on DNN-based HSI classification methods from the following two perspectives.

A. Handcrafted Deep Neural Networks for HSI Classification

By means of its strong ability in feature extraction and classification, DNN has widely been researched for image classification. In this direction, CNN-based classification methods are most widely studied in the literature [13]–[15]. Specifically, the basic components of a CNN include multiple convolutional layers for extracting features, pooling layers for downsampling, and fully connected layers for feature mapping [26], [29], [35]. Envious of the outstanding performance of CNNs in normal image classification, researchers have also been devoted to designing CNNs for HSI classification [37]–[42].

At first, researchers separately took advantage of the spectral and spatial features for HSI classification. For example, the work of [37] first uses the principal component analysis method (PCA) to reduce the dimensionality of the spectral features so as to compress HSI data into a few principal component bands and then performs 2D-CNN to obtain the spatial details. Due to the utilization of PCA for band selection and the separate use of the spectral–spatial features, this method cannot fully utilize the contextual information and thus its classification performance is limited. To alleviate this limitation, researchers turn to exploiting the joint spectral–spatial features for HSI classification. For instance, the work of [38] devised a 3D convolutional neural network (3D-CNN) to extract the deep spectral–spatial-combined features for HSI classification. Furthermore, to avoid overfitting, the work of [39] investigated a few regularization strategies for 3D-CNN based on the joint spectral–spatial features. To fully exploit the spectral and spatial information embedded in pixels, the work of [40] proposed a deep CNN to learn pixel-pair features (CNN-PPF). In this algorithm, for each pixel, pixel pairs are first constructed by combining the center pixel and each of the surrounding pixels, and then the label of the pixel is determined by these pixel pairs classified by the trained CNN. Furthermore, a diverse region-based CNN (DRCNN) [41] was proposed by merging a diverse set of discriminative appearance factors to learn contextual interactional features. Specifically, this method exhibits spatial–spectral context sensitivity and thus obtains very promising classification performance.

As the research on CNN-based HSI classification goes, it is found that the handcrafted CNN networks become deeper and deeper. As a result, not only does the optimization complexity of the handcrafted models increase, but also the difficulty and instability of parameter optimization of the models arise. These two limitations might degrade the classification ability of CNNs. To alleviate the issue, the work of [42] combined the residual learning [43] to design a multiscale convolutional filter bank for CNN to explore local spectral–spatial contextual interactions, and form a joint feature map to feed a fully convolutional network to predict the label of each pixel. Furthermore, the work of [44] proposed an end-to-end spectral–spatial residual network (SSRN), which uses spectral residual and spatial residual blocks to continuously learn discriminative features from abundant spectral signatures and spatial contexts. Recently, to further improve the classification performance, attention mechanisms have been combined into CNNs [45], [46] for HSI classification. For instance, in [45], an end-to-end residual spectral–spatial attention network (RSSAN) was proposed by designing multiple attention modules. Specifically, a spectral attention module was designed for spectral band selection, a spatial attention module was devised for adaptive selection of spatial information, two attention modules were developed for adaptive feature refinement in spectral–spatial feature learning, and a sequential spectral–spatial attention module was put forward to avoid overfitting.

Except for CNN-based HSI classification methods, researchers have also developed other handcrafted DNNs for HSI classification. For example, the work of [47] combined the deep belief network (DBN) and the conjugate gradient (CG) update algorithm to extract the spectral features and pixel-centric spectral block features for HSI classification. In [46], Li *et al.* proposed a spatial attention-driven recurrent feedback convolutional neural network (SARFNN) method for HSI classification. In particular, this algorithm adopts the recurrent feedback and attention mechanism structures to learn the spatial features with a 2D model and capture the joint spatial–spectral features with a 3D model. In [48], a multitask generative adversarial network (MTGAN) method was developed for HSI classification by exploiting the rich information from unlabeled samples. Specifically, in this method, a generator network was devised to undertake the reconstruction task and the classification task, while a discriminator network was constructed to identify whether the input sample comes from the real distribution or the reconstructed one. In [49], a nonlocal capsule network (NLCapsNet) was proposed for HSI classification by introducing a nonlocal block of attention into the capsule neural network (CapsNet). By capturing the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

spatial relationships among HSI features, this algorithm could isolate invalid information, and at the same time, consolidate valid information. To jointly leverage the merits of the graph convolutional network (GCN) and CNN in extracting different types of HSI features, in [50], Hong *et al.* first put forward a new minibatch GCN, and then devised three fusion strategies (namely, additive fusion, elementwise multiplicative fusion, and concatenation fusion) to fuse the proposed GCN and CNN for HSI classification. Besides, in [51], Liu *et al.* also proposed a CNN-enhanced GCN (CEGCN) for HSI classification. Specifically, in this algorithm, CNN performs feature learning on small-scale regular regions, while GCN extracts features on large-scale irregular regions. In this way of cooperation, the proposed algorithm could generate complementary spectral–spatial features at both pixel and superpixel levels. Recently, taking advantage of the merit of transformers, the authors in [52] adapted the bidirectional encoder representations from transformers (BERT) for HSI classification, leading to a transformer-based classification method, named HSI-BERT. With the multihead self-attention (MHSA) mechanism, this algorithm could capture the global dependence among pixels, and thus obtain very discriminative features with rich spatial–spectral information. Furthermore, in [53], Hong *et al.* devised a novel backbone network along with transformers, named SpectralFormer, to learn spectrally local sequence information from neighboring bands of HSI to improve the HSI classification accuracy.

The above handcrafted DNN methods have shown promising performance on HSI classification. However, the architectures of all these methods are meticulously handcrafted, which heavily depends on the professional skills and experience of researchers on architecture designs of networks [25], [27], [35]. At the same time, the deployment of hyperparameters in the designed architectures also requires a lot of tedious trial-and-error fine-tuning experiments, which is extremely time-consuming and energy-consuming. With these two limitations, the usability of the above HSI classification methods is limited.

B. EA-Based Automatic CNN for Image Classification

To improve the usability of CNN-based classification methods, researchers attempt to employ optimization algorithms to automatically optimize the CNN architecture, which leads to a new emerging topic, called NAS [25]–[27]. As shown in Fig. 1, given the predefined search space, NAS methods iteratively search the optimal architecture automatically based on feedback evaluated on the training dataset and the validation dataset [25].

As a kind of metaheuristic algorithms, EAs maintain a population of candidate solutions to iteratively search the solution space [35]. Due to their strong global search ability and independence on the mathematic properties (e.g., continuous, convex, and differentiable) of optimization problems, they are often selected as the optimization algorithm in Fig. 1 to automatically search the optimal CNN architecture. In this direction, GAs are commonly adopted [28], [33].

As shown in Fig. 2, GA maintains multiple chromosomes, each of which represents a feasible solution, to iteratively find the global optimal solution. Three main operators, namely

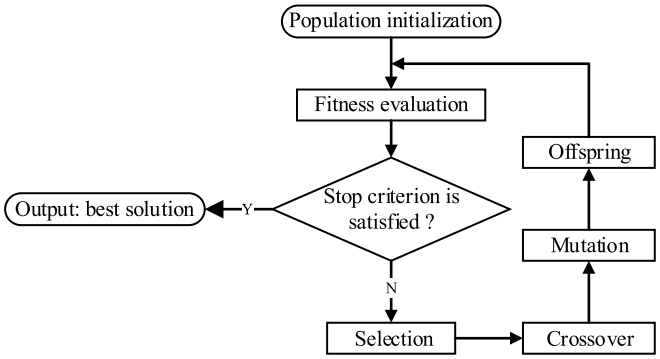


Fig. 2. General flowchart of GA.

selection, crossover, and mutation, are utilized to generate new solutions (namely offspring). With the iterations of the three operators, the found optimal solution becomes better and better and then at the end of the iteration, a promising solution closed to the ideal solution can be obtained [29], [32].

From different perspectives, researchers have developed different NAS methods based on GA to seek the optimal CNN architectures [30], [32], [33]. In a broad sense, existing GA-based NAS methods can be classified into two categories: 1) micro methods, which aim to optimize some certain components of the architecture with the overall structure fixed [33]–[35], and 2) macro methods, which optimize the overall architecture based on predefined components [28]–[30].

Micro optimization methods are usually dedicated to finding effective components from the search space and filling them into the handcrafted network templates designed via professional knowledge and experience. In [33], a genetic CNN was proposed. Specifically, it first divides a CNN into different stages manually. Then, it encodes a network structure into a fixed-length binary string with each element representing the internode connection. Its aim is to utilize GA to seek the optimal connections between internal nodes (namely convolution operations) in different stages. To speed up the evolution of GA, the work of [34] proposed an efficient evolution method of the neural architecture (EENA) with the guidance of experience gained in the prior learning. In [35], a fast evolutionary neural architecture search framework based on sampled training of the parent individuals along with a node inheritance strategy (termed SI-EvoNAS) to start the evolution of GA. Then, it constructs a standard version of the network (termed SI-EvoNet-S) to build the entire neural network by stacking the optimized blocks sequentially with residual connections.

Different from the above micro optimization methods, macro optimization methods focus on the optimization of the overall CNN architecture by searching for the optimal combinations of predefined components. In [29], an automatically evolving CNN (AE-CNN) was proposed to optimize the depth of the CNN architecture and the associated components in each layer. Specifically, this algorithm first designs two basic blocks based on the ResNet block [43] and the DenseNet block [54], and then devises a variable-length encoding scheme for GA

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

to search the optimal depth and the components of the CNN architecture. Besides, in [30], an efficient continuous evolutionary approach named continuous evolution architecture search (CARS) was devised for searching neural networks. Specifically, this algorithm constructs an archive to store the trained parameters of the supernets found in each generation and then the newly generated architectures by GA share parameters within the supernet set to start the training instead of training from scratch.

Although the abovementioned NAS methods have shown promising performance on normal image classification, they are hardly utilized for HSI classification directly due to the difference between HSIs and normal images. As far as we know, there are very few studies [36] on NAS methods for HSI classification. In [36], Chen *et al.* proposed 3D-Auto-CNN, a NAS method based on gradient descent optimization. Specifically, this method focuses on searching for two desired cells to stack the network and it actually is a micro optimization NAS method. Therefore, existing NAS methods for HSI classification optimize the architecture from the micro space, namely optimizing the components with the overall structure fixed. These methods still heavily rely on abundant experience and knowledge of users on architecture design, which restricts the use of these methods in real-world applications.

To alleviate this issue, this article proposes an EB-CNN for HSI classification, which simultaneously optimizes the components and the overall structure of CNN based on predesigned basic blocks. In this way, the dependence on the expert knowledge and experience on CNNs could be largely reduced and thus the usability of the proposed method is improved.

III. Proposed Algorithm

To reduce the dependence on professional experience and knowledge of users on CNNs and parameter fine-tuning to a large extent, this article proposes an EB-CNN to simultaneously and automatically optimize the components and the overall structure (the width and the depth) of CNNs for HSI classification.

A. Overall Framework

The overall framework of the proposed EB-CNN is given in Algorithm 1. From this algorithm, it is found that the framework contains the following four main steps:

Step 1: Initialization of the Population(line 1). In this step, N chromosomes, which form the population $\boldsymbol{P}_g$ of GA, are first randomly built based on the encoding scheme and the predefined search space to be introduced next.

Step 2: Fitness Evaluation(line 2). To evaluate the fitness, each chromosome in $\boldsymbol{P}_g$ is first decoded into a CNN and then the constructed CNN is trained on the training dataset. Subsequently, the trained CNN is evaluated on the validation set and the validation accuracy is taken as the final fitness of the associated chromosome.

Step 3: **Offspring Generation and Evaluation** (lines 4 and 5). In this step, first, the binary tournament algorithm [55] is used to sequentially select promising chromosomes as parents, and then, the modified crossover and mutation operators are executed on the selected parents to produce N offspring $\boldsymbol{Q}_g$. Subsequently, each offspring is decoded into a CNN network, trained on the training dataset and evaluated on the validation dataset to obtain its fitness $\boldsymbol{F_{og}}$.

Step 4: **Environmental selection**(line 6). In this step, the offspring population $\boldsymbol{Q}_g$ and the parent population $\boldsymbol{P}_g$ are combined to select N chromosomes to form the parents $\boldsymbol{P}_{g+1}$ for the next generation. In addition, in this step, the historically found global best solution denoted as ***gbest*** is also updated if the best solution found in this generation is better. Furthermore, if ***gbest*** is not in $\boldsymbol{P}_{g+1}$, then the worst solution in $\boldsymbol{P}_{g+1}$ will be replaced by ***gbest***.

Algorithm 1 Procedure of EB-CNN

Input: Maximum number of generations G, Population size N, Maximum chromosome length L, Predefined basic blocks $\boldsymbol{B}$, and Dataset $\boldsymbol{S}$.

1: **Initialization**: $\boldsymbol{P}_g \leftarrow$ Initialize the population randomly based on the input parameters N, L, $\boldsymbol{B}$, $\boldsymbol{S}$ and set $g = 0$;
2: **Evaluation**: $\boldsymbol{F_{pg}} \leftarrow$ Decode each initialized chromosome into a CNN network, and evaluate its fitness on the given dataset via training the network and obtaining the validation accuracy;
3: **While** ($g \leq G$)
4: **Offspring generation**: $\boldsymbol{Q}_g \leftarrow$ Use the binary tournament selection and the proposed crossover and mutation operators to generate offspring;
5: **Evaluation**: $\boldsymbol{F_{og}} \leftarrow$ Evaluate the fitness of $\boldsymbol{Q}_g$ in the same way in Step 2;
6: **Selection**: ***gbest***, $\boldsymbol{P}_{g+1} \leftarrow$ From $\boldsymbol{P}_g \cup \boldsymbol{Q}_g$, obtain the optimal chromosome and generate a new population ($\boldsymbol{P}_{g+1}$) by the roulette selection;
7: $g \leftarrow g+1$;
8: **End While**;
Output: The best chromosome ***gbest***.

Finally, Step 3 and Step 4 are iterated until the termination condition is met. Then, at the end of the algorithm, the found global optimal chromosome ***gbest*** is output and decoded into a CNN as the final architecture.

B. Designed Basic Blocks

Generally, a CNN is composed of multiple convolutional layers for feature extraction, several pooling layers for feature downsampling, and a fully connected layer for feature mapping [26], [29], [35]. The former two kinds of layers usually intertwine with each other, while the last kind of layers is generally placed at the end of the architecture [28], [30], [34]. Based on this observation, the fully connected layer is utilized at the end of the optimal architecture found by the proposed EB-CNN by default and thus is not included in the predefined search space. As a consequence, this article only needs to construct search space for the convolutional layers and the pooling layers.

The key to designing the search space for the two layers is to keep the search space as small as possible and, at the same

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

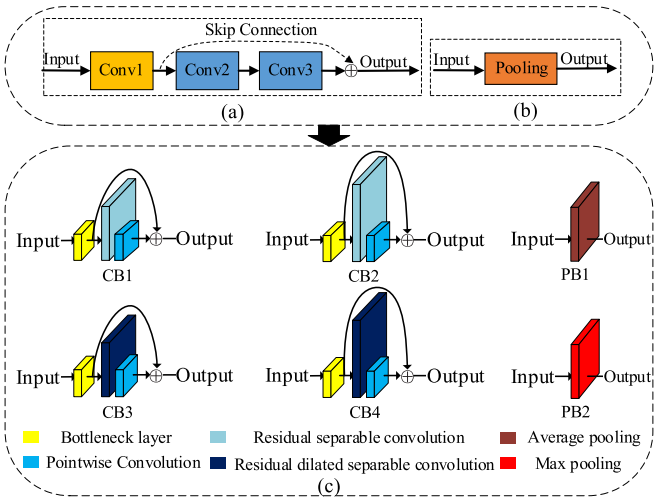


Fig. 3. Designed two novel basic blocks and predefined search space. The structure of (a) CB and (b) PB. (c) Predefined search space.

time, keep the generalization of the constructed architecture as good as possible. Inspired from the block-based search space design methods in [26], [29], and [32], which reduce the search space to a large extent, compared with other space design approaches [28], [30], [35], this article designs two types of basic blocks to form the micro search space for the abovementioned two layers.

As shown in Fig. 3(a), for the convolutional layer, this article first designs a general convolutional block (CB) for spatial and spectral feature extraction of HSIs. From this figure, it is found that the general CB consists of three convolutional layers and a jump connection. The jump connection is also called the skip connection [56]. Such a short-circuit connection could transfer the input information directly to the output across multiple layers. It is mainly used to alleviate the degradation issue which might occur during the training of CNN. In the literature, the typical ResNet [43] benefits from this short-circuit connection to achieve promising performance. Based on this observation, it is adopted in the designed CB.

The first convolutional layer (conv1) identified in yellow in Fig. 3(a) is the input bottleneck convolutional layer. Its kernel size is usually fixed as 1×1 as commonly set in the literature [29], [32], [43]. The second and the third layers (conv2 and conv3) marked in blue in Fig. 3(a) are the convolutional layers used to extract informative features. Conv2 adopts the depthwise convolution [57] to extract various informative features of HSIs, while conv3 utilizes the pointwise convolution [57] to further cooperate with conv2 to make the extracted features as informative as possible. In the designed CB, the kernel size of conv3 is usually set as 1×1 as in the literature [26], [36], [57].

As for conv2, this article adopts two kinds of depthwise convolutions, namely residual separable convolution [57] and residual dilated separable convolution [58], to form two kinds of conv2. It should be mentioned that the residual dilated separable convolution is adopted here because it has been proven to systematically aggregate multiscale contextual information without losing resolution [26], [35], [36]. Since HSIs contain rich spatial and spectral information, the utilization of the residual dilated separable convolution can further extract discriminative features, prevent the loss of detailed information embedded in HSIs, and reduce the parameters of components in the search space. With respect to the kernel size of conv2, this article predefines two kinds of settings, namely, 3×3 and 5×5. These two kinds of kernel size settings are selected because they are commonly used to constitute classical promising architecture of handcrafted CNNs in the literature [26], [29], [35]. Consequently, there are four kinds of settings for conv2, namely, 3×3 residual separable convolution, 5×5 residual separable convolution, 3×3 residual dilated separable convolution, and 5×5 residual dilated separable convolution.

TABLE I

PREDEFINED SIX BLOCKS INCLUDING FOUR DIFFERENT CBS AND TWO DIFFERENT PBS

Operation Type	Kernel Size	Code	Name
Residual separable convolution	3	10	CB1
Residual separable convolution	5	11	CB2
Residual dilated separable convolution	3	12	CB3
Residual dilated separable convolution	5	13	CB4
Average pooling	3	20	PB1
Max pooling	3	21	PB2

Based on the above predefinition, this article presets four kinds of basic CBs, as shown in Fig. 3(c). From this figure, it is found that the main difference among these four CBs lies in the setting of conv2. Therefore, the settings of conv2 are used to identify these CBs, as shown in Table I. These CBs form the search space for convolutional layers.

As shown in Fig. 3(b), for the pooling layer, this article designs a general pooling block (PB) for feature downsampling. Since the main function of PB is to compress features by removing less informative information, this article presets the kernel size as 3×3 and the step size of PB as 2×2 as commonly set in existing studies [29], [32], [35]. In this way, the size of extracted features in CBs is halved after the pooling layer. As for the pooling type, this article utilizes the maximum pooling and the average pooling to configure the designed PB. As a result, as shown in Fig. 3(c), two kinds of PBs are constructed to form the search space for the pooling layers.

Together, the predefined search space for the convolutional layers and the pooling layers is shown in Table I. To facilitate the encoding of chromosomes in GA to be elucidated next, the coding of CBs and PBs is also presented in this table. It deserves attention that based on these predefined blocks, the optimization of the CNN architecture can be seen as a combinatorial optimization problem with respect to these blocks, which is an NP-hard problem. Optimizing such a problem involves optimizing the number of the associated blocks and the arrangement of these blocks. To solve this problem effectively and efficiently, this article adopts GA to automatically optimize the combinations to find an optimal CNN architecture.

C. Encoding and Decoding Schemes of the Chromosome

The first step of using GA to automatically find an optimal architecture of a CNN is to encode the chromosome via utilizing the designed PBs and CBs and then decode the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

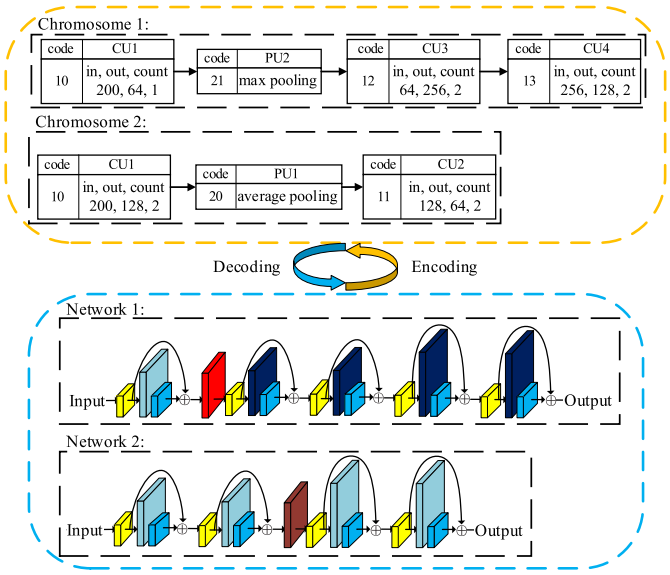


Fig. 4. Encoding and decoding scheme of EB-CNN.

chromosome to a CNN to evaluate its fitness. Therefore, this section depicts the encoding and decoding schemes for GA.

First, we define a new concept called the convolution unit (CU), which contains a number (denoted by *count*) of the same type of CBs. Therefore, for the above designed four kinds of CBs (CB1, CB2, CB3, and CB4), four different CUs (CU1, CU2, CU3, and CU4) can be obtained, with each containing several associated CBs. Based on Table I, each CU could be encoded as a quadruple (*code*, *in*, *out*, *count*), where "*code*" is the code of the associated CB, as shown in Table I, "*in*" is the number of input channels of the CU, which is in accordance with the dimensionality of the input data or the number of the output channels of the first CU ahead of it, "*out*" is the number of the output channels of the CU, which is initialized randomly in the initialization stage or evolved during the evolution, and "*count*" is the number of the associated CBs.

Similarly, for the PBs, we define the pooling unit (PU), which also contains a number of the same type of PBs. For the designed two kinds of PBs (PB1 and PB2), two different PUs (PU1 and PU2) could be obtained, with each containing several associated PBs. However, it should be mentioned that PB is mainly for feature downsampling and, therefore, in the literature, it is less likely that several consecutive PBs are utilized to constitute the architecture of a CNN. This is because several consecutive PBs continuously reduce the number of features exponentially, which might lead to serious loss of discriminative information and thus result in poor performance of the associated CNN. Moreover, PB in different locations can make the architecture more flexible. Therefore, in each kind of PUs, the number of PBs is set as 1. As a consequence, each PU can be directly encoded as the code of the associated PB.

Subsequently, these CUs and PUs are utilized to encode each chromosome. Specifically, as shown in Fig. 4, each component of a chromosome is constituted by one PU or CU. As for the length of a chromosome, inspired by existing studies [29], [31], [32], this article devises a flexible strategy to allow different chromosomes to have different lengths. With this mechanism, different chromosomes can represent different CNNs with different depths. In the initialization stage, the length of each chromosome is randomly sampled in a given range. Then, during the evolution, the length of each chromosome is self-adaptively determined by the operators in GA and thus the depth of the CNN architecture can be optimized.

To visually and vividly understand the encoding scheme of the chromosome, an example is shown in Fig. 4. In the top part of the figure, two encoded chromosomes are presented. The first chromosome can be expressed as [(10, 200, 64, 1), (21), (12, 64, 256, 2), (13, 256, 128, 2)]. This indicates the architecture decoded by this chromosome contains 1 CB1, 1 PB2, 2 CB3, and 2 CB4. Similarly, the second chromosome can be expressed as [(10, 200, 128, 2), (20), (11, 128, 64, 2)]. This indicates the architecture decoded by this chromosome contains 2 CB1, 1 PB1, and 2 CB2. The lengths of these two chromosomes are 4 and 3, respectively.

Based on such an encoding scheme, it is natural to decode each chromosome into a CNN, as shown in Fig. 4. However, it deserves attention that as aforementioned in Section III-B, the fully connected layer is utilized at the end of the architecture by default. Therefore, each chromosome in GA only contains the information about the convolution layers and the pooling layers. As a result, the complete architecture of the CNN decoded by the associated chromosome should contain a fully connected layer at the end.

To summarize, from Fig. 4, it is found that the devised encoding scheme has two main advantages. 1) As shown in Fig. 4, it is simple and intuitive to decode each chromosome into a CNN based on this encoding. 2) It is very flexible to optimize the depth of the architecture because, on the one hand, different chromosomes could have different lengths; on the other hand, different CUs could have different numbers of CBs.

Based on this encoding scheme, the overall architecture of a CNN can be simultaneously optimized by GA from three perspectives. 1) In view of the arrangement of different units, the combination of PUs and CUs can be optimized by the evolution of the chromosomes. 2) In terms of the depth of the architecture, it can be optimized in two ways, namely the number of blocks in CUs and the length of the chromosome. 3) With respect to the width of each layer in the architecture, it can be optimized by the number of channels in the associated CU.

D. Evolutionary Processing

After the above preparation, this section elaborates the evolutionary process of GA in detail.

1) Population Initialization: Algorithm 2 displays the pseudocode of the population initialization. Given the population size N, the maximum length of chromosomes L, the predefined units $\boldsymbol{U}$, and the dimension size of input data D, the maximum number (denoted by M) of PUs in a chromosome is first calculated (line 2). It is noted that in the predefined PBs, the stride size is set as 2×2, and PB is mainly used for feature downsampling. Therefore, in the CNN architecture,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Population Initialization

Input: Population size N, Maximum chromosome length L, Predefined basic units $\boldsymbol{U}$, and Dimension size of input data D.

1: $\boldsymbol{P}_0 \leftarrow \emptyset$;
2: Set the maximum number of PUs as $M \leftarrow \mathbf{log}_2 D$;
3: $i \leftarrow 0$;
4: **While** ($i < N$) **do**
5: $h \leftarrow$Initialize the chromosome length randomly within [2, L];
6: $\boldsymbol{T} \leftarrow$Create h empty unit nodes for the ith chromosome;
7: Set the number of PUs as PU_num = 0;
8: **For each empty node in** $\boldsymbol{T}$ **do**
9: **If** PU_num $\geq M$ **Then** // the number of PUs in a chromosome should not exceed M;
10: *code* $\leftarrow$ Randomly select a unit from the predefined CUs;
11: **Else**
12: **If** it is the first node **then** // the first node should not be PU
13: *code* $\leftarrow$ Randomly select a unit from the predefined CUs;
14: **Else**
15: *code* $\leftarrow$ Randomly select a unit from the predefined units $\boldsymbol{U}$ (CUs and PUs);
16: **End If**
17: **End If**
18: **If** the node is CU **Then**
19: *in*, *out*, *count* $\leftarrow$Initialize the number of channels (*in* and *out*) and the number (*count*) of CBs;
20: **Else**
21: PU_num = PU_num+1;
22: **End If**
23: **End For**
24: $\boldsymbol{P}_0 \leftarrow \boldsymbol{P}_0 \cup \boldsymbol{T}$;
25: $i \leftarrow i+1$;
26: **End While**;

Output: The initialized population $\boldsymbol{P}_0$

Algorithm 3 Fitness Evaluation

Input: Population $\boldsymbol{P}_g$, Training and validation datasets namely $\boldsymbol{D}_{train}$, and $\boldsymbol{D}_{valid}$.

1: **For** each chromosome (the ith one) $\boldsymbol{T}$ in $\boldsymbol{P}_g$ **do**
2: Decode the ith chromosome into a ***CNN*** based on the information encoded in $\boldsymbol{T}$;
3: Train the constructed ***CNN*** on $\boldsymbol{D}_{train}$;
4: $F_i \leftarrow$ Set the validation accuracy of the trained ***CNN*** on $\boldsymbol{D}_{valid}$ as the fitness of the ith chromosome;
5: **End For**

Output: The fitness of the population $\boldsymbol{P}_g$

the number of PBs is at most $\log_2(D)$. As a result, in the following initialization process, if the number of PUs in a chromosome equals to M, the rest uninitialized components of the chromosome can only be set as CUs (lines 9–10).

In the initialization process of a chromosome, the length (denoted by h) of the chromosome is first uniformly and randomly sampled from [2, L] (line 5). Then, each component is randomly initialized from the predefined units, namely CUs and PUs. It should be mentioned that the pooling layer is generally after the convolution layer. Therefore, in each chromosome, the first component should not be a PU (line 12) and thus can only be initialized by a CU. As for other components, they can be initialized either by a CU or by a PU as long as the number of PUs (line 15) in the currently initialized chromosome is smaller than M.

After the code initialization (namely the type of units) of each component, the components initialized with CUs are further deployed some key parameters (lines 18 and 19). Specifically, the number of the input channels ("*in*") of the CU equals to either the dimension size of input data if it is the first node in the chromosome, or the number of the output channels of the first CU ahead of this one. For example, in the first chromosome (chromosome 1) in Fig. 4, parameter "*in*" of the first CU (namely CU1) equals to the dimension size of the input data (namely 200), while parameter "*in*" of the second CU (namely CU3) equals to the number of the output channels of the first CU (namely CU1), which is 64, and that parameter of the third CU (namely CU4) is 256, which is the number of the output channels of the second CU (CU3).

Different from the number of the input channels ("*in*"), the number of the output channels ("*out*") of the CU in the associated component is randomly initialized within [64, 256]. At last, the number of the associated CBs ("*count*") in each CU is also randomly initialized within [3, 10].

After all the above initialization, the construction of a complete chromosome ($\boldsymbol{T}$) is completed and then it is added into the initial population $\boldsymbol{P}_0$ (line 24). Then, the program continues to construct another chromosome (line 25) in the same way until N chromosomes are all completely constructed.

2) Fitness Evaluation: As shown in Fig. 2, after the initialization of the population, it is the fitness evaluation process of the population, which is displayed in Algorithm 3.

To evaluate the quality of the architecture represented by one chromosome, the first step is to decode the chromosome to a CNN (line 2). As mentioned above in Section III-C, due to the direct encoding scheme of the chromosome, the construction of a CNN based on one chromosome is natural by cascading the associated CBs or PBs in each component of the chromosome.

After the decoding of a chromosome, the constructed CNN is trained on the given training dataset ($\boldsymbol{D}_{\text{train}}$). Then, the trained CNN is tested on the given validation dataset ($\boldsymbol{D}_{\text{valid}}$). The validation accuracy of the trained CNN is set as the fitness of the associated chromosome.

It is worth mentioning that the algorithm evaluates all chromosomes under the same environment, like the same parameter settings for training the weights. To quantify the classification performance of each chromosome (CNN), the overall accuracy (OA) on the validation set $\boldsymbol{D}_{\text{valid}}$ is employed as the performance measure. OA is the ratio of the number of correctly labeled hyperspectral pixels to the total number of hyperspectral pixels on $\boldsymbol{D}_{\text{valid}}$. Meanwhile, the proposed method uses the Adam trainer [59] to train each constructed CNN. In addition, the softmax regression classifier is utilized at the end of each constructed CNN to classify each pixel.

3) Selection: As shown in Fig. 2, the first operator of GA is to select parents for next generation. To this end, Algorithm 4 shows the environment selection of the proposed EB-CNN.

First of all, the global best solution ***gbest*** found currently is first identified from the parents $\boldsymbol{P}_g$ and the offspring $\boldsymbol{Q}_g$ (line 2). Then, since the evaluation of a chromosome needs to train the associated CNN, which is very time-consuming, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4 Environmental Selection

Input: Population size N, Parent population $\boldsymbol{P}_g$, and Offspring $\boldsymbol{Q}_g$.

1: $\boldsymbol{P}_{g+1} \leftarrow \emptyset$;
2: ***gbest*** $\leftarrow$ Obtain the best chromosome from $\boldsymbol{P}_g \cup \boldsymbol{Q}_g$;
3: **For** $i \leftarrow 1$ **to** N **do**
4: $\quad \boldsymbol{q}_i \leftarrow$ Use the roulette selection method to randomly select a chromosome from $\boldsymbol{P}_g \cup \boldsymbol{Q}_g$;
5: $\quad \boldsymbol{P}_{g+1} \leftarrow \boldsymbol{P}_{g+1} \cup \boldsymbol{q}_i$;
6: **End For**;
7: **If** ***gbest*** is not in $\boldsymbol{P}_{g+1}$ **then**
8: $\quad$ Use ***gbest*** to replace the worst chromosome in $\boldsymbol{P}_{g+1}$;
9: **End If**

Output: The population $\boldsymbol{P}_{g+1}$ for next generation of evolution

Algorithm 5 Offspring Generation

Input: Parent population $\boldsymbol{P}_g$, Crossover probability μ, and Mutation probability Υ.

1: $\boldsymbol{Q}_g \leftarrow \emptyset$;
2: **While** ($|\boldsymbol{Q}_g| \neq |\boldsymbol{P}_g|$) **do**
3: $\quad$ **For** $i \leftarrow 1$ **to**2 **do** // binary tournament selection
4: $\quad\quad$ Randomly select two chromosomes $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ from $\boldsymbol{P}_g$;
5: $\quad\quad \boldsymbol{p}_i \leftarrow$ Select the better one between $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$;
6: $\quad$ **End For**
7: $\quad$ **If** rand(0,1) $\leq \mu$ **then**
8: $\quad\quad$ Randomly sample a cross point (c_1) for $\boldsymbol{p}_1$ and count the number of PUs in the two segments p_{11} and p_{12};
9: $\quad\quad$ Randomly sample a cross point (c_2) for $\boldsymbol{p}_2$ and count the number of PUs in the two segments p_{21} and p_{22};
10: $\quad\quad$ **While** $p_{11} + p_{22} >$ M or $p_{21} + p_{12} > M$ **do**
11: $\quad\quad\quad$ Resample a cross point (c_2) for $\boldsymbol{p}_2$ and count the number of PUs in the two segments p_{21} and p_{22};
12: $\quad\quad$ **End While**
13: $\quad\quad$ Exchange information of $\boldsymbol{p}_1$ starting from c_1+1 to the end of $\boldsymbol{p}_1$ and that of $\boldsymbol{p}_2$ starting from c_2+1 to the end of $\boldsymbol{p}_2$ to generate two offspring $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$;
14: $\quad\quad \boldsymbol{Q}_g \leftarrow \boldsymbol{Q}_g \cup (\boldsymbol{e}_1 \cup \boldsymbol{e}_2)$;
15: $\quad$ **Else**
16: $\quad\quad \boldsymbol{Q}_g \leftarrow \boldsymbol{Q}_g \cup (\boldsymbol{p}_1 \cup \boldsymbol{p}_2)$;
17: $\quad$ **End If**
18: **End While**;
19: **For** each chromosome $\boldsymbol{p}$ in $\boldsymbol{Q}_g$ **do**
20: $\quad$ **If** rand(0,1) $\leq \Upsilon$ **then**
21: $\quad\quad$ Randomly select a mutation strategy for $\boldsymbol{p}$;
22: $\quad\quad$ Execute the associated mutation on $\boldsymbol{p}$;
23: $\quad$ **End If**
24: **End For**

Output: The generated offspring $\boldsymbol{Q}_g$.

population evolved by GA should converge fast to a promising solution. To this end, this article combines the parents $\boldsymbol{P}_g$ and the offspring $\boldsymbol{Q}_g$ as a whole set $\boldsymbol{P}_g \cup \boldsymbol{Q}_g$ to select promising individuals for next generation. To preserve high diversity to escape from local areas, this article utilizes the roulette selection algorithm to randomly select N individuals to form the parents $\boldsymbol{P}_{g+1}$ for next generation from $\boldsymbol{P}_g \cup \boldsymbol{Q}_g$ instead of using the absolutely best N individuals (lines 3–6). At last, the historically best solution generally preserves the most promising evolutionary information to guide the evolution of the population. Therefore, to make full use of such evolutionary information to accelerate the search of the population, EB-CNN adopts an elite strategy [60] to explicitly keep ***gbest*** into the next round of evolution (lines 7–9).

Comprehensively speaking, with the above selection mechanism, the proposed EB-CNN is expected to compromise high search diversity and fast convergence speed well to search the solution space to find a promising architecture.

4) Crossover: After the selection of the parents for next generation, it goes to the offspring generation. The first step to generate offspring is to use the crossover operator to exchange information between two selected parents. The general process of the crossover operation is shown in lines 2–18 in Algorithm 5.

Specifically, as shown in lines 3–6, the first step is to use the binary tournament selection strategy to randomly select two different parents for information exchange. Then, as shown in line 7, if the randomly generated value within [0, 1] is smaller than the preset crossover probability, the two randomly selected parents exchange their information based on the one-point crossover (lines 8–13). In this way, two offspring are generated and are stored into the new population (line 14). If the randomly generated value is larger than the crossover probability, the selected two parents do not exchange information and thus are directly stored into the new population (line 16).

During the crossover between two parents, three issues need to be paid careful attention. 1) Since this article adopts the flexible encoding scheme as shown in Section III-C, which allows different chromosomes to have different lengths, the cross points randomly selected for the two parents are not allele. Suppose the lengths of the two parents are h_1 and h_2, respectively. Then, the cross point (c_1) of the first parent ($\boldsymbol{p}_1$) is randomly sampled from [1, h_1-1], while that (c_2) of the second parent ($\boldsymbol{p}_2$) is randomly sampled from [1, h_2-1]. It is noticed that it is likely that $c_1 \neq c_2$. 2) It also deserves attention that after the crossover, the length of one offspring might be larger than the maximum length L. In this situation, the algorithm resamples the crossover points for the two selected parents until the lengths of the two generated offspring are both smaller than L. To avoid wasting too much time in trying to find the appropriate crossover points, the algorithm stops executing the crossover operator on the two selected parents if it takes five trials and still cannot sample suitable crossover points for the two selected parents. In this situation, the two parents are directly stored into the new population $\boldsymbol{Q}_g$. 3) As aforementioned, the number of PUs in a chromosome is at most $M = \log_2(D)$, where D is the dimension size of the input data. The crossover operation between the two parents may lead to invalid chromosomes, where the number of PUs exceeds M. To avoid this situation, this article first randomly samples the cross point (c_1) for the first parent ($\boldsymbol{p}_1$) and then the numbers of PUs in the two segments of $\boldsymbol{p}_1$ are counted (denoted by p_{11} and p_{12}, respectively). Subsequently, for the second parent ($\boldsymbol{p}_2$), this article first randomly samples a cross point (c_2) and counts the numbers of PUs in the two segments (denoted by p_{21} and p_{22}, respectively). If either the sum of p_{11} and p_{22} or the sum of p_{21} and p_{12} is larger than M,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

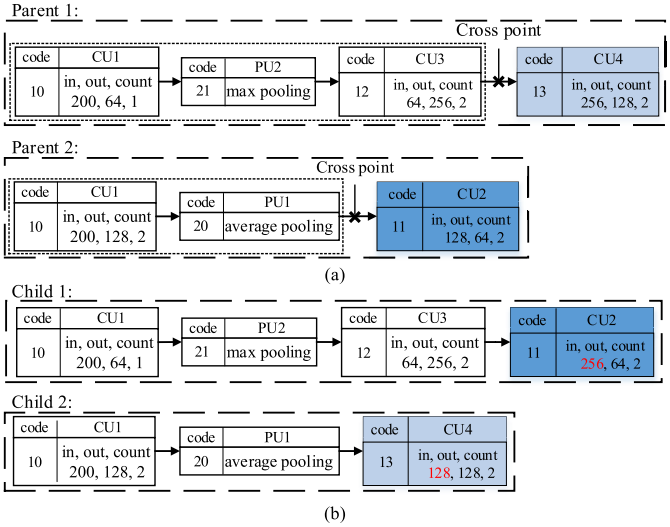


Fig. 5. Process of the crossover operator. (a) Before crossover. (b) After crossover.

the cross point of $\boldsymbol{p}_2$ is resampled until both the sum of p_{11} and p_{22} and the sum of p_{21} and p_{12} are smaller than M.

Once the cross points of the two parents are confirmed, the crossover operation is conducted on the two parents by exchanging information of $\boldsymbol{p}_1$ starting from c_1+1 to the end of $\boldsymbol{p}_1$ and that of $\boldsymbol{p}_2$ starting from c_2+1 to the end of $\boldsymbol{p}_2$ to generate two offspring $\boldsymbol{e}_1$ and $\boldsymbol{e}_2$ (line 13). After the information exchange, if the component after the cross point of $\boldsymbol{e}_1$ or $\boldsymbol{e}_2$ is a type of CUs, the number (*in*) of the input channels of this CU might be inconsistent with the number of the output channels (*out*) of the first CU ahead of this component. In this situation, "*in*" of this CU is corrected to equal to "*out*" of the first CU ahead of it.

To visually understand the crossover process, this article takes the two chromosomes in Fig. 5 as examples to execute the crossover operation. As shown in Fig. 5, the randomly sampled cross point for parent 1 is $c_1 = 3$ and that for parent 2 is $c_2 = 2$. Then, the numbers of PUs of the two segments in parent 1 and parent 2 are counted, namely $p_{11} = 1$, $p_{12} = 0$, $p_{21} = 1$, and $p_{22} = 0$, respectively. Given that the maximum number of PUs in a chromosome is 4, namely $M = 4$, $p_{11} + p_{22} = 1 < M$, and $p_{21} + p_{12} = 1 < M$. Therefore, these two parents exchange their information starting from the first component after the cross points c_1 and c_2 to the end of the associated chromosomes as marked in blue in Fig. 5. After the exchange, two offspring are generated (denoted by child 1 and child 2, respectively).

After the information exchange, it is found that the number of the input channels (*in*) of the component CU2 in child 1 is 128, which is not consistent with the number of the output channels (*out*) of CU3 (the first CU ahead of CU2), which is 256. Therefore, "*in*" of CU2 is modified to be 256, as shown in Fig. 5. Similarly, for child 2, "*in*" of the component CU4 is 256, which is not consistent with "*out*" of CU1 (the first CU ahead of CU4), which is 128. Hence, "*in*" of CU4 is modified to be 128, as shown in Fig. 5.

5) Mutation: After the crossover operation, it goes to the mutation operation. In GA, the mutation operator plays a key role in introducing new values for the chromosomes that do not appear in the parents. Therefore, the mutation operator is mainly for diversity enhancement for the population.

As shown in lines 19–24 in Algorithm 5, for each chromosome in the generated offspring after crossover, if the uniformly generated random value within [0, 1] is smaller than the preset mutation probability, the associated chromosome will be mutated based on a specified mutation operation.

As for the mutation strategy, inspired from existing studies on GA-based NAS [29], [32], this article adapts three kinds of mutation operations, namely, adding, removing, and modifying, to the proposed EB-CNN. Besides, to improve the search effectiveness and efficiency of GA, this article further develops a new mutation strategy named swapping to mutate the components of a chromosome. As a result, there are 4 types of mutations in EB-CNN, which are listed as follows:

a) *Adding.* Literally, this operation is to add a new component for the associated chromosome, and thus will increase the length of the chromosome by 1. Therefore, the following issues deserve careful attention. 1) As mentioned in Section III-B, the maximum length of a chromosome is L. Therefore, if the length of the chromosome is already L, the adding mutation is not conducted. 2) Once a component is added to the chromosome, such a component can be initialized as either a CU or a PU. However, it should be noticed that as aforementioned, the maximum number of PUs in a chromosome is M. Consequently, if the number of PUs is already M in a chromosome, the added component can only be initialized as a CU. 3) If the added component can be initialized as either a PU or a CU, the mutation strategy first randomly initializes the type of the added component (namely PU or CU) with equal probabilities. Then, if it is initialized as PU, the strategy will further randomly select a configuration from the predefined PUs (PU1 and PU2) with equal probabilities. Similarly, if it is initialized as CU, the strategy will also randomly select a configuration from the predefined CUs (CU1, CU2, CU3, and CU4). 4) After the initialization of the added component, the mutation strategy will randomly select an insertion point between 1 and the length of the associated chromosome and then the component is added after the insertion point of the chromosome. 5) After the insertion, if the added component is a type of CUs, some key parameters of the CU should be initialized. Since each CU is encoded as a quadruple (*code*, *in*, *out*, and *count*), "*code*" is set as the code of the CU, "*in*" is set as the number of the output channels ("*out*") of the first CU ahead of the added component, and "out" is set as the number of the input channels ("*in*") of the first CU after the added component. If there is no CU after the component, "out" is randomly set within [64, 256]. As for the number of CBs "count," it is randomly set within [3, 10].

To visually understand the process of the adding mutation, Fig. 6 presents an example. First, suppose that the maximum length (L) of a chromosome is 12 and the maximum number ($\boldsymbol{M}$) of PUs in a chromosome is 4. The length of the chromosome in Fig. 6(a) is 4 and the number of PUs is 1. Therefore, the adding mutation can be operated on this chromosome and the added component can be either a PU or a CU.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

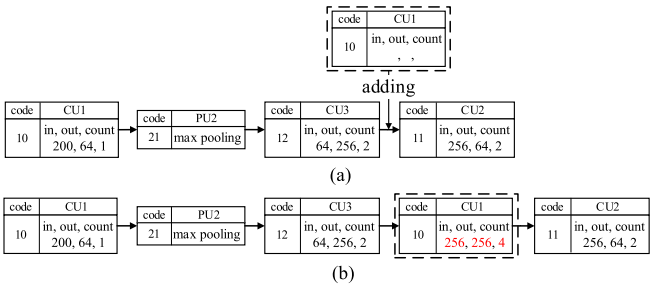


Fig. 6. Process of the adding operator with channel correction. (a) Before adding. (b) After adding.

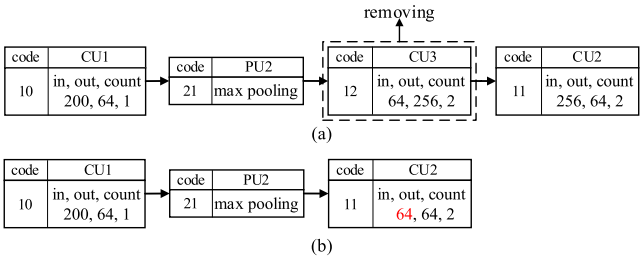


Fig. 7. Process of the removing operator with channel correction. (a) Before removing. (b) After removing.

As shown in Fig. 6, after the random initialization, the added component is set as a type of CUs (CU1). Then, an insertion point is randomly sampled between 1 and the length of the chromosome (4), which is 3, as shown in Fig. 6. Subsequently, the added component is inserted into the chromosome. At last, the key parameters of the added CU are set as (10, 256, 256, 4), where "10" is the code of CU1, the first "256" is the number of the input channels in the added CU1, which is determined by the number of the output channels in CU3 (ahead of the added component), the second "256" is the number of the output channels in the added CU1, which is determined by the number of the input channels in CU2 (after the added component), and "4" is the number of CB1s in the added CU1, which is randomly generated.

b) Removing. Literally, this operation is to remove a component from a chromosome, which decreases the length of the chromosome by 1. The first step of this operation is to randomly select a component from the second component to the last component of the associated chromosome. It should be noticed that the first component of a chromosome cannot be removed due to the protection of the validity of the chromosome. If the removed component is a PU, it is directly deleted. However, if the removed component is a CU, it may occur that the number of the input channels (*in*) of the first CU after the removed CU is not consistent with the number of the output channels (*out*) of the first CU ahead of the removed component. In this situation, this article corrects "*in*" of the first CU after the removed component to be the "*out*" of the first CU ahead of the removed component.

To visually understand the process of the removing mutation, Fig. 7 shows an example. In this example, the component CU3 is selected to remove. After deleting this component, the number of the input channels (*in*) of CU2 (the first CU after the removed CU3), which is 256, is not consistent with the number of the output channels (*out*) of CU1 (the first CU ahead of the removed CU3), which is 64. Hence, it is corrected to be 64.

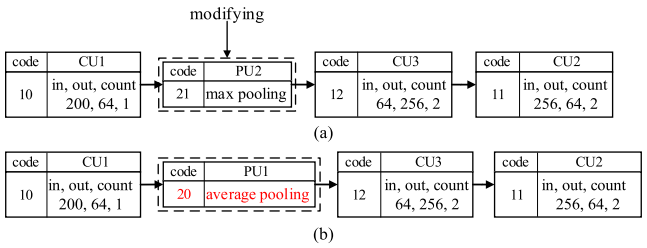


Fig. 8. Process of the modifying operator. (a) Before modifying. (b) After modifying.

c) Modifying. This mutation operation aims to modify the associated parameters of a component in a chromosome. Specifically, the first step of this operation is to randomly select a component from the chromosome for modification. Then, if the selected component is a PU (suppose it is PU1), since there are only two predefined types of PUs, this component will be modified to be another PU (namely PU2). However, if the selected component is a CU, since a CU is encoded by a quadruple (*code*, *in*, *out*, and *count*), there are four ways to modify the component. a) Modifying the code of the CU with other parameters unchanged. This modification is to change the type of the CU. It is conducted by randomly selecting a type of CU from the remaining three CUs. b) Modifying the number of the input channels (*in*) of the CU with other parameters unchanged. It should be noticed that the number of the input channels of the first CU is determined by the input data and thus this modification cannot be executed on the first CU of the chromosome. As for the intermediate CUs, if this modification is performed, the number of the output channels (*out*) of the first CU ahead of the modified component should also be accordingly changed. This modification is executed by randomly sampling a value within [64, 256] to replace "*in*" of the selected CU and "*out*" of the first CU ahead of the selected component. c) Modifying the number of the output channels (*out*) of the CU with other parameters unchanged. This modification can be executed on any CU. It also deserves attention that if this modification is performed, the number of the input channels (*in*) of the first CU after the modified component should also be accordingly changed. This modification is executed by randomly sampling a value within [64, 256] to replace "*out*" of the selected CU and "*in*" of the first CU after the selected component. d) Modifying the number of the associated CBs (*count*) of the CU with other parameters unchanged. This modification can be executed on any CU. It is executed by randomly sampling a value within [3, 10] to replace "*count*" of the selected CU.

Fig. 8 displays an example to show the process of the modifying mutation. In this example, a PU is selected for modification. Since there are only two types of PUs, the selected PU2 is modified to be PU1 in this example.

d) Swapping. This mutation is newly designed in this article for the proposed EB-CNN. It aims to swap two components in a chromosome. The first step of this mutation is to randomly select two components from the chromosome. Then, the two

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

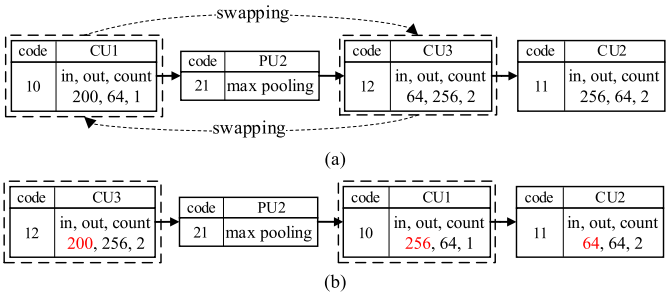


Fig. 9. Process of the swapping operator with channel correction. (a) Before swapping. (b) After swapping.

components are swapped. However, some issues need to be noticed. 1) As previously mentioned, the first component of a chromosome cannot be a PU. Therefore, if the first component is selected to swap with a PU, such a swap is not valid. In this situation, either a new component is randomly selected to replace the first component for swapping or a CU component is randomly selected to replace the selected PU for swapping. 2) If the selected two components are both PUs, the two components are directly swapped without any correction. 3) If the selected two components are both CUs or one is a CU and another is a PU, after the swapping, the number of the input channels (*in*) and the number of the output channels (*out*) of the related CUs need to be corrected, so that these two parameters of the associated CUs are consistent with their neighbor CUs.

Fig. 9 presents an example of the swapping mutation. In this example, the first component (CU1) and the third component (CU3) are selected to swap. After the swapping, the number of the input channels (*in*) of CU3 is corrected to be 200 as the dimension size of the input data, while that of CU1 is corrected to be 256 to be consistent with the number of the output channels (*out*) of CU3. At the same time, "*in*" of CU2 also needs to be corrected as 64 to be consistent with "*out*" of CU1.

Remark: The swapping operator is newly designed in this article to accelerate the evolution of the chromosomes to the promising architecture. Compared with the former three mutation strategies, the swapping mutation neither changes the length of the chromosome, nor changes the types of CUs or PUs in the components of the chromosome. That is to say, the swapping operation does not introduce new components or new values for the chromosome but only changes the positions of two components. Therefore, the swapping mutation can be considered as a local search method and thus is beneficial for improving the convergence of the population.

To visually understand the advantage of the swapping mutation, Fig. 10 presents an example. Given a chromosome A shown in Fig. 10(a), it needs at least four continuous steps to transform A to the target superior chromosome B shown in Fig. 10(d) if only adopting the adding, removing, and modifying mutation operations, as shown in Fig. 10(b). However, if utilizing the swapping mutation, it only needs one step to transform A to B.

However, it should be noticed that as aforementioned, on the one hand, each chromosome in the generated offspring

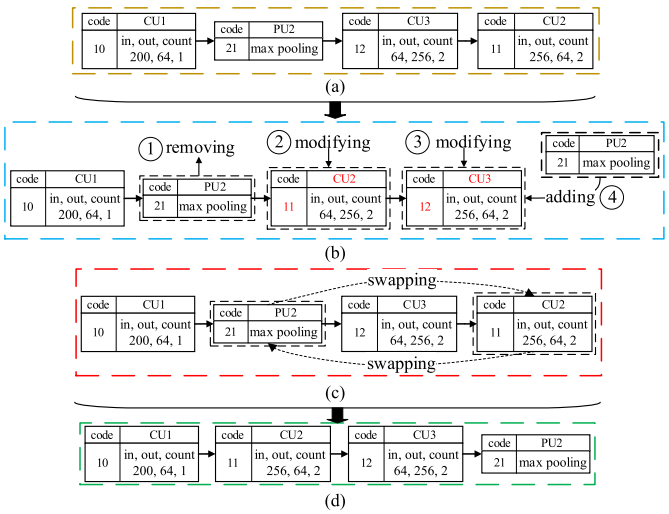


Fig. 10. Two transformation processes from chromosome A to chromosome B. (a) Chromosome A. (b) Chromosome A with four mutation operations: 1→2→3→4. (c) Chromosome A with swapping. (d) Target superior chromosome B.

after crossover performs the mutation operator with a small probability; on the other hand, even though a chromosome conducts the mutation operation, only one type of mutations is randomly selected to mutate the chromosome. As a consequence, it is likely that the number of generations needed to evolve chromosome A to chromosome B is far from 4. From this perspective, it is found that compared with the former three mutation operations, the proposed swapping operator could accelerate the evolution of the population, which is not only beneficial for fast convergence to promising solutions, but also makes great contributions to saving computational resources, such as the number of fitness evaluations, which is very expensive for NAS methods.

With the abovementioned evolutionary processes, the proposed EB-CNN iterates generation by generation to find the optimal CNN architecture for HSI classification.

E. Difference Between EB-CNN and Existing Automatic Methods

As far as we know, there are very few works on utilizing NAS methods for HSI classification. The most similar work with the proposed EB-CNN is 3D-Auto-CNN [36], which is a gradient descent-based NAS method for HSI classification. Similar to the proposed EB-CNN, 3D-Auto-CNN also adopts block-based search methods to find the optimal structure. However, it assumes that the overall CNN architecture is fixed and only focuses on searching for optimal cells for each component of the fixed architecture based on predefined cells. In addition, the number of internal nodes in the predefined cells and the connection between the cells are also fixed by manual design. Therefore, 3D-Auto-CNN can be considered as a semiautomatic NAS method, where expert knowledge and experience are still heavily involved to carefully design the overall network.

Different from 3D-Auto-CNN, the proposed EB-CNN preserves the following advantages:

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NINE STATE-OF-THE-ARTS COMPARED ALGORITHMS

Algorithm	Category
SVM-RBF	Traditional classification method
CD-CNN	Hand-crafted architecture
SSRN	Hand-crafted architecture
CSMS-SSRN	Hand-crafted architecture
DBDA	Hand-crafted architecture
CEGCN	Hand-crafted architecture
HSI-BERT	Hand-crafted architecture
NLCapsNet	Hand-crafted architecture
3D-Auto-CNN	Semi-automatic NAS method

1) First of all, EB-CNN is a completely automatic NAS method. It cannot only simultaneously optimize the depth and the width of the architecture, but also optimizes the components of the architecture. The former is realized by optimizing the number of channels (*in* and *out*), the number of blocks in the adopted units (*count*), and the length of the chromosome. The latter is realized by optimizing the combinations of the predefined units.
2) Second, it is flexible to evolve the chromosomes due to the designed encoding scheme. Different from most existing GA-based NAS methods [33]–[35], where all chromosomes have the same fixed length, the proposed EB-CNN allows different chromosomes to have different lengths and such a length is evolved during optimization.
3) At last, compared with most existing NAS methods [33], [35], [36], the involved expert knowledge and experience is reduced to a large extent. The only involved expert knowledge and experience on CNN is to design the search space, namely to predefine the blocks. At present, designing search space for NAS methods is indispensable and thus is involved in nearly all NAS methods. Such expert knowledge and experience is far less than that on the overall network designs. Therefore, the proposed EB-CNN preserves better feasibility and usability.

IV. EXPERIMENTS

This section conducts extensive experiments to verify the effectiveness of the proposed EB-CNN. Specifically, experimental settings including compared algorithms and the used datasets are first introduced in Section IV-A. Then, in Section IV-B, the parameter settings of EB-CNN are explained, following which are the comparison results and analysis between EB-CNN and the compared peer algorithms in Section IV-C. At last, Section IV-D conducts experiments to take deep investigation on the optimization of EB-CNN to find the optimal architecture.

A. Experimental Settings

First, to comprehensively demonstrate the effectiveness of the proposed EB-CNN, this article selects nine state-of-the-art HSI classification methods to make extensive comparisons, which can be divided into three categories, as listed in Table II. These algorithms are the nonlinear SVM based on Gaussian radial basis kernel function (SVM-RBF) [20], the contextual deep CNN (CD-CNN) [42], SSRN [44], CSMS-SSRN [24], the double-branch dual-attention mechanism network (DBDA) [14], CEGCN [51], HSI-BERT [52], NLCapsNet [49], and 3D-Auto-CNN [36]. SVM-RBF [20] is a representative traditional HSI classification algorithm. CD-CNN [42], SSRN [44], CSMS-SSRN [24], DBDA [14], CEGCN [51], HSI-BERT [52], and NLCapsNet [49] are seven deep-learning-based HSI classification algorithms. The architectures of these seven methods are all elaborately designed by researchers with professional knowledge and experience on neural architecture design. 3D-Auto-CNN [36] is, as far as we know, the first NAS method for HSI classification.

Second, to validate the effectiveness of the proposed EB-CNN, this article selects five commonly used HSI datasets, namely, Indian Pine (IN), Salinas Valley in California (SA), University of Pavia (UP), Houston University 2013 (HU2013) [61], and Houston University 2018 (HU2018) [62], to conduct extensive experiments. The brief information of the five datasets is listed in Table SI and Table SII in the supplementary material, respectively.

To evaluate the fitness of chromosomes in GA and validate the proposed EB-CNN, following the selection rules in [15], [24], and [45], this article randomly selects nearly 5%, 5%, and 90% of the total pixels in the IN dataset as the training dataset, the validation dataset, and the testing dataset, respectively. While for the SA and UP datasets, the selection ratios of the training dataset, the validation dataset, and the testing dataset are 3%, 3%, and 94% of the total pixels, respectively. As for the HU2013 [61] and HU2018 [62] dataset, we follow the selection rule in [6] to select the pixels for the training dataset, and then, we randomly select 20% and 80% of the remaining pixels as the validation set and the testing set, respectively. Specifically, for the five datasets, the numbers of pixels belonging to different classes in the training dataset, the validation dataset, and the testing dataset are listed in Table SI and Table SII in the supplementary material, respectively.

B. Parameter Settings

The proposed EB-CNN is to automatically find an optimal CNN architecture for HSI classification. Therefore, there is no need to set hyperparameters related to CNN, which usually requires extensive and tedious human effort to fine-tune. However, to let the proposed EB-CNN work efficiently, some configurations also need to be preset.

First, for the flexible encoding scheme, the maximum length of a chromosome is set as 12, which means that there are at most 12 units in a chromosome. To significantly increase the depth of CNN, the number of CBs in each CU is randomly sampled within [3, 10]. As for the width of each convolution layer, namely, the number of the output channels in each CU, it is randomly sampled within [64, 256]. In addition, since a CNN contains at least one convolution layer and a pooling layer, the number of CUs and the number of PUs initially are at least 1 in each chromosome, respectively.

Second, for GA, the number of evolved chromosomes is set as 20 and the maximum number of generations is set as 10, respectively. As for the crossover and mutation probabilities,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CLASSIFICATION RESULTS (%) ON IN DATASET

Algorithm / Index	SVM-RBF	CD-CNN	SSRN	CSMS-SSRN	DBDA	CEGCN	HSI-BERT	NLCapsNet	3D-Auto-CNN	EB-CNN
OA	73.82	72.07	94.02	93.16	96.34	96.61	86.32	92.46	96.67	**96.77**
AA	72.49	70.29	92.53	93.04	95.27	91.01	74.81	92.72	**95.29**	95.21
Kappa	69.89	67.72	93.17	93.90	95.84	96.22	84.37	91.39	96.20	**96.31**
1	35.64	37.52	60.66	90.00	**100.00**	97.13	43.17	86.36	90.91	96.33
2	62.50	72.62	96.02	95.02	**98.38**	97.77	80.83	86.38	98.10	97.01
3	68.19	50.37	92.96	91.33	94.72	**97.34**	81.49	88.92	97.21	96.61
4	52.33	62.12	99.47	93.45	**100.00**	93.55	54.41	92.20	99.05	94.79
5	86.41	94.80	**99.00**	97.30	98.26	97.51	86.43	91.16	98.77	97.02
6	85.47	90.23	98.79	97.18	98.02	**99.93**	94.99	93.76	93.93	97.00
7	75.75	56.95	**100.00**	83.33	63.16	93.70	36.76	84.73	**100.00**	85.07
8	89.27	88.67	98.87	93.07	**100.00**	99.95	95.53	99.01	**100.00**	99.63
9	59.79	45.31	75.00	**100.00**	**100.00**	25.54	40.38	95.83	**100.00**	**100.00**
10	68.84	61.02	90.22	84.77	89.80	94.33	82.50	87.77	**97.44**	96.88
11	70.44	65.81	91.44	96.16	**98.93**	98.89	89.49	91.44	97.92	97.78
12	61.37	48.15	93.82	92.75	96.23	**98.31**	76.86	87.11	89.06	92.33
13	87.07	93.13	**100.00**	**100.00**	99.44	**100.00**	97.16	93.48	**100.00**	95.85
14	89.59	88.72	94.20	95.94	93.43	**99.83**	98.63	95.70	97.50	98.22
15	68.52	76.98	**96.62**	91.32	93.88	95.80	88.37	89.91	93.24	95.15
16	98.65	92.24	93.41	89.01	**100.00**	**100.00**	77.46	87.83	71.59	84.15

TABLE IV
CLASSIFICATION RESULTS (%) ON SA DATASET

Algorithm / Index	SVM-RBF	CD-CNN	SSRN	CSMS-SSRN	DBDA	CEGCN	HSI-BERT	NLCapsNet	3D-Auto-CNN	EB-CNN
OA	91.56	90.62	94.98	97.57	99.08	**99.78**	94.15	99.30	99.63	**99.78**
AA	95.25	94.21	97.13	98.97	99.45	**99.71**	95.62	99.50	99.35	99.64
Kappa	90.59	89.56	94.40	97.30	98.98	99.74	93.49	99.26	99.58	**99.76**
1	99.76	87.43	**100.00**	99.89	**100.00**	**100.00**	97.55	98.98	**100.00**	99.96
2	99.32	93.15	**100.00**	**100.00**	**100.00**	**100.00**	97.49	99.46	99.74	**100.00**
3	94.94	96.60	73.24	**100.00**	**100.00**	**100.00**	95.24	99.64	99.36	99.85
4	97.99	99.11	99.01	98.87	**99.27**	99.88	98.65	98.28	98.56	97.89
5	98.31	98.74	99.94	**100.00**	99.84	99.82	98.51	98.79	98.22	99.59
6	99.93	99.49	99.95	**100.00**	99.99	**100.00**	99.84	**100.00**	99.97	99.93
7	99.49	97.13	**100.00**	**100.00**	**100.00**	99.96	99.65	**100.00**	**100.00**	99.98
8	79.40	84.79	86.01	94.19	99.23	**99.97**	94.55	98.16	99.90	99.88
9	99.06	99.77	99.66	99.91	99.85	**100.00**	99.55	98.96	99.83	99.95
10	93.41	95.67	98.13	99.35	98.72	99.71	94.81	98.30	**100.00**	99.76
11	95.51	84.08	**100.00**	99.90	**100.00**	99.96	97.10	99.87	**100.00**	99.93
12	97.59	98.92	**100.00**	**100.00**	**100.00**	**100.00**	99.61	98.73	98.68	99.21
13	96.75	96.94	99.65	99.77	99.94	99.93	98.41	**100.00**	97.13	99.34
14	95.65	97.41	99.80	**100.00**	98.73	98.55	96.87	99.70	98.51	99.23
15	78.58	78.73	98.67	91.64	95.69	99.67	76.37	96.72	99.72	**99.81**
16	98.21	99.40	**100.00**	**100.00**	**100.00**	99.68	97.37	**100.00**	**100.00**	99.86

they are set as 0.9 and 0.2, respectively, as commonly set in GA-based NAS methods [29], [32], [63].

Third, for the fitness evaluation of a chromosome, namely the training and validation of CNN, this article adopts $32 \times 32 \times B$ neighborhood pixels as the input data for the IN, UP, and SA datasets as adopted in [36], and $16 \times 16 \times B$ neighborhood pixels for the HU2013 and HU2018 datasets as adopted in [6], where B is the number of the HSI bands in the associated datasets. In addition, L_2 regularization is used in the constructed CNN by each chromosome to avoid over-fitting as commonly adopted in the literature [36], [39]. The weight decay in optimizing the weight by back propagation (BP) algorithm is set to 0.0003 as commonly set in the literature [36], [44], [45]. The constructed CNN by each chromosome is trained using the Adam trainer [59] with 120 epochs and 32 batches. The learning rate of the Adam trainer is initially set as 0.01 and then is dynamically adjusted as the epoch increases.

At last, it is worth mentioning that all algorithms in the experiments adopt the same experimental settings. To make fair and comprehensive comparison, all algorithms are run 10 independent times and compared from three perspectives, namely the OA, the average accuracy (AA), and Kappa coefficients (Kappa), on each dataset. In addition, all experiments are performed on a graphics processing unit (GPU) server with a 64-bit Ubuntu system and 2 NVIDIA RTX 2080Ti GPU.

C. Overall Comparison With Compared Methods

In this section, extensive experiments are conducted on the five dataset to compare the proposed EB-CNN with the selected compared algorithms. Particularly, Tables III–VII show the comparison results among these algorithms on the IN, SA, UP, HU2013, and HU2018 datasets, respectively. In these tables, the first three rows present the overall performance of different algorithms on the whole dataset. The following rows display the classification results of different algorithms on each label. The best value is highlighted in bold in the tables.

As shown in Table III, the comparison results between EB-CNN and the compared algorithms on the IN dataset

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CLASSIFICATION RESULTS (%) ON UP DATASET

Index \ Algorithm	SVM-RBF	CD-CNN	SSRN	CSMS-SSRN	DBDA	CEGCN	HSI-BERT	NLCapsNet	3D-Auto-CNN	EB-CNN
OA	92.19	91.11	99.41	99.34	99.40	**99.50**	96.03	97.79	99.33	99.41
AA	91.32	89.61	99.33	99.22	99.25	**99.48**	94.21	97.35	98.48	98.92
Kappa	89.59	88.19	99.22	99.13	99.24	**99.54**	94.78	97.17	99.11	99.22
1	92.51	91.70	99.82	99.49	99.40	**99.97**	97.40	95.30	99.45	99.43
2	94.50	96.34	99.70	99.76	**99.86**	99.85	99.66	98.31	99.84	99.84
3	81.74	76.38	98.36	98.19	97.62	98.81	80.89	91.62	98.29	**99.19**
4	95.37	97.30	**100.00**	99.96	98.50	98.35	95.73	96.82	97.24	97.99
5	98.66	99.27	**100.00**	99.92	99.84	**100.00**	99.85	98.60	99.13	98.81
6	91.07	86.66	99.70	**100.00**	99.81	**100.00**	95.67	96.02	99.79	**100.00**
7	84.99	78.42	**100.00**	**100.00**	96.71	99.96	74.65	97.19	99.68	99.06
8	83.03	81.82	96.36	95.70	96.22	99.34	91.74	95.19	**99.79**	98.84
9	90.56	98.64	**100.00**	**100.00**	98.32	**100.00**	95.86	96.50	93.06	97.27

TABLE VI
CLASSIFICATION RESULTS (%) ON HOUSTON 2013 DATASET

Index \ Algorithm	SVM-RBF	CD-CNN	SSRN	CSMS-SSRN	DBDA	CEGCN	HSI-BERT	NLCapsNet	3D-Auto-CNN	EB-CNN
OA	76.51	80.23	83.75	80.44	83.61	81.71	82.27	66.05	84.14	**84.15**
AA	78.82	81.46	82.77	83.66	83.01	83.09	84.02	68.07	86.25	**87.65**
Kappa	74.93	78.56	82.36	80.44	82.22	80.18	80.84	63.22	**84.19**	82.34
1	83.00	99.85	98.59	96.20	96.51	98.39	82.49	78.69	99.65	**99.88**
2	**98.40**	88.28	96.59	97.88	97.27	92.11	82.91	76.49	90.18	96.42
3	99.59	50.13	72.94	94.73	88.86	**99.60**	89.01	67.39	81.42	66.32
4	**98.48**	93.11	95.20	97.89	74.89	96.78	92.64	85.28	91.74	97.07
5	97.82	96.83	92.16	97.50	**100.00**	97.73	99.50	83.77	90.62	**100.00**
6	90.91	70.45	57.83	11.75	57.71	**95.10**	83.57	43.01	91.40	69.14
7	90.39	62.27	87.48	84.88	81.94	57.28	83.81	80.68	**95.30**	89.02
8	40.46	77.60	54.59	89.21	82.04	69.09	68.26	42.86	60.97	**89.51**
9	41.93	78.43	79.74	85.47	77.12	53.92	82.97	78.37	**94.16**	88.32
10	62.64	73.20	95.58	83.04	**97.94**	77.41	54.03	62.03	80.15	90.97
11	75.43	63.82	85.29	81.80	64.01	84.91	75.70	81.01	85.28	**97.31**
12	60.04	91.94	**93.39**	92.22	90.15	77.23	85.66	40.97	70.59	88.89
13	49.47	84.82	78.08	74.04	47.72	50.88	86.14	60.49	82.90	**95.24**
14	98.79	97.20	74.11	92.50	91.30	98.38	98.06	86.73	98.01	**99.60**
15	97.46	93.92	80.01	97.78	97.44	**98.52**	95.54	79.01	81.40	87.06

TABLE VII
CLASSIFICATION RESULTS (%) ON HOUSTON 2018 DATASET

Index \ Algorithm	SVM-RBF	CD-CNN	SSRN	CSMS-SSRN	DBDA	CEGCN	HSI-BERT	NLCapsNet	3D-Auto-CNN	EB-CNN
OA	54.07	45.89	63.55	61.62	**64.05**	60.80	63.11	32.75	61.30	63.57
AA	47.73	33.84	52.67	60.75	52.06	59.64	**63.42**	25.81	50.31	47.75
Kappa	43.48	38.15	58.09	53.35	**61.01**	58.65	54.75	26.44	54.47	54.82
1	81.01	60.41	82.80	82.36	88.54	61.30	63.78	28.24	**88.84**	51.50
2	**91.93**	77.09	83.14	86.13	79.74	61.53	84.32	72.15	83.49	74.62
3	94.32	10.82	95.05	93.58	12.77	63.71	**100.00**	6.78	19.74	21.87
4	84.36	68.55	88.73	92.34	85.21	61.38	**95.62**	5.92	88.38	81.22
5	34.55	20.83	58.51	48.82	50.89	**61.38**	51.28	12.04	21.89	24.05
6	21.84	3.41	52.26	33.92	56.15	**62.48**	31.36	2.26	22.51	14.75
7	90.33	0.93	22.06	**100.00**	2.34	59.02	30.25	0.45	39.69	72.01
8	44.08	40.03	48.79	52.58	48.91	22.56	**84.67**	42.60	54.42	50.69
9	83.29	87.53	92.75	94.01	**95.09**	33.08	60.52	86.22	93.42	90.64
10	34.44	34.68	46.53	37.82	49.67	14.39	**58.14**	32.67	44.91	41.99
11	23.24	33.87	40.05	36.16	40.96	9.32	**67.05**	20.68	40.78	45.93
12	0.42	2.63	9.59	6.33	0.73	**19.99**	6.17	0.86	19.91	3.79
13	55.77	40.87	53.57	61.33	**63.49**	11.73	40.92	30.91	63.20	55.71
14	59.39	38.89	47.25	57.59	39.91	59.60	51.17	39.92	44.48	**90.64**
15	53.73	36.34	50.73	63.98	**83.02**	60.30	65.88	18.07	65.25	31.53
16	32.02	17.41	**68.79**	44.34	60.33	58.90	62.51	27.01	52.28	51.44
17	9.56	0.17	7.10	**100.00**	26.78	58.56	**100.00**	1.29	45.85	1.34
18	34.59	39.02	27.24	15.96	52.19	**58.60**	50.76	11.51	40.53	33.92
19	19.73	38.72	38.71	76.57	58.27	60.38	86.32	9.86	20.77	**87.16**
20	30.63	27.22	40.01	31.21	46.13	60.88	**73.62**	14.04	55.95	40.07

are summarized as follows. 1) From the perspective of the overall performance, as shown in the first three rows, the proposed EB-CNN achieves the best overall performance with respect to "OA" and "Kappa." Regarding "AA," EB-CNN obtains considerably competitive performance with DBDA and 3D-Auto-CNN. 2) Specifically, compared with SVM-RBF and CD-CNN, the developed EB-CNN obtains significantly better performance by improving "OA," "AA," and "Kappa" with at least 23%, 23%, and 27%, respectively. In comparison with HSI-BERT, EB-CNN is much superior by improving the three indicators with nearly 10%, 11%, and 12%, respectively. Compared with SSRN, CSMS-SSRN, and NLCapsNet, EB-CNN outperforms them by improving the three indicators with at least 2%, 2%, and 3%, respectively. Competed with DBDA, CEGCN, and 3D-Auto-CNN, EB-CNN achieves slightly better performance. 3) As for the classification performance on each

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

label, it is found that EB-CNN obtains higher accuracy than 92% on 14 labels, and achieves lower accuracy than 84% on no labels. However, SVM-RBF, CD-CNN, HSI-BERT, and NLCapsNet achieve higher accuracy than 92% on at most six labels. Although DBDA, CEGCN, and 3D-Auto-CNN achieve higher accuracy than 92% on 14, 15, and 13 labels, respectively, all of them obtain lower accuracy than 84% on 1 label.

As shown in Table IV, the comparison results among the ten algorithms on the SA dataset are summarized as follows. 1) As shown in the first three rows, EB-CNN achieves the best overall performance with respect to "OA" and "Kappa," while CEGCN achieves the best overall performance with respect to "OA" and "AA." 2) Specifically, compared with SVM-RBF and CD-CNN, the developed EB-CNN performs significantly better by improving "OA," "AA," and "Kappa" with at least 8%, 4%, and 9%, respectively. In competition with SSRN, CSMS-SSRN, and HSI-BERT, EB-CNN presents superiority to them by improving the three indicators with at least 2%, 1%, and 2%, respectively. In comparison with DBDA, CEGCN, NLCapsNet, and 3D-Auto-CNN, EB-CNN achieves very competitive performance with them. 3) In terms of the classification performance on each label, except for the 4th label, the classification accuracy of all the other class labels obtained by EB-CNN is at least 99.21%. However, regarding the three competitive methods, namely DBDA, CEGCN, NLCapsNet, and 3D-Auto-CNN, their classification accuracy is lower than 99% on 3, 1, 8, and 5 labels, respectively.

As shown in Table V, the comparison results on the UP dataset are summarized as follows: 1) From the perspective of the overall performance, as shown in the first three rows, the proposed EB-CNN achieves significantly better performance than SVM-RBF and CD-CNN by improving "OA," "AA," and "Kappa" with at least 7%, 7%, and 10%, respectively. In comparison with HSI-BERT and NLCapsNet, EB-CNN outperforms them by improving the three indicators with at least 2%, 1%, and 2%, respectively. In competition with the other five compared methods, EB-CNN obtains very similar performance with them. 2) As for the classification accuracy on each label, with respect to the six competitive algorithms, namely EB-CNN, 3D-Auto-CNN, CEGCN, DBDA, CSMS-SSRN, and SSRN, EB-CNN and CEGCN obtain at least 97% classification accuracy on all labels. However, 3D-Auto-CNN, DBDA, CSMS-SSRN, and SSRN attain less than 97% classification accuracy on at least 1 label.

As shown in Table VI, the comparison results on the HU2013 dataset are summarized as follows. 1) From the perspective of the overall performance, as shown in the first three rows, the proposed EB-CNN achieves the best overall performance with respect to "OA" and "AA." Regarding "Kappa," EB-CNN obtains considerably competitive performance with SSRN, DBDA, and 3D-Auto-CNN. 2) Specifically, compared with SVM-RBF, CD-CNN and NLCapsNet, the proposed EB-CNN obtains better performance by improving "OA," "AA," and "Kappa" with at least 4%, 6%, and 4%, respectively. In comparison with CSMS-SSRN, CEGCN, and HSI-BERT, EB-CNN is much superior to them by improving the three indicators with at least 2%, 3%, and 2%, respectively. Competed with 3D-Auto-CNN, DBDA, and SSRN, EB-CNN achieves slightly better performance. 3) As for the classification performance on each label, EB-CNN achieves the best classification results on six labels, while the other algorithms obtain the best results on at most three labels. In particular, the classification accuracy of EB-CNN is larger than 87% on 13 labels, while the 9 compared algorithms obtain more than 87% classification accuracy at most nine labels.

As shown in Table VII, the comparison results on the HU2018 dataset are summarized as follows. 1) From the perspective of the overall performance, as shown in the first three rows, the proposed EB-CNN achieves significantly better performance than SVM-RBF, CD-CNN, and NLCapsNet by improving "OA" and "Kappa" with at least 9%, and 11%, respectively. In competition with the other six compared methods, EB-CNN presents competitive performance with them. 2) As for the classification performance on each label, it is found that EB-CNN achieves the best performance on two labels (the 14th and 19th labels). It obtains very low classification accuracy on the 12th and 17th labels, but still performs better than two compared algorithms.

Subsequently, to visually observe the difference between the proposed EB-CNN and the compared methods with respect to the classification performance on a whole HSI image, this article utilizes the final trained models of different algorithms in one random run to classify the whole HSI images in the five datasets. Figs. S1-S5 in the supplementary material show the classification maps obtained by different methods on the IN, SA, UP, HU2013, and HU2018 datasets, respectively.

From these figures, at the first glimpse, comparing the classification map of each algorithm with the true map shown in Table SI and Table SII in the supplementary material, we find that on the IN dataset, the classification maps of EB-CNN, 3D-Auto-CNN, DBDA, CEGCN, and NLCapsNet are much closer to the true map than the other compared algorithms. On the SA dataset, EB-CNN, 3D-Auto-CNN, and CEGCN obtain much closer classification maps to the true map than the other compared algorithms. On the UP dataset, EB-CNN, 3D-Auto-CNN, and CSMS-SSRN achieve much closer classification maps to the true map than the other seven algorithms. On the HU2013 set, there is no clear difference among the classification maps of all algorithms. On the HU2018 set, the classification map of EB-CNN is much closer to the true map than all the compared algorithms. As a whole, it is concluded that the classification map of EB-CNN is very close to the true map on all datasets. This is because instead of using the same handcrafted architecture in CD-CNN, SSRN, CSMS-SSRN, DBDA, CE-GCN, HSI-BERT, and NLCapsNet to deal with all the five datasets, EB-CNN automatically finds the optimal CNN architecture for each dataset. Generally speaking, such architectures are different for different datasets and can well fit for the associated datasets. This is the particular and unique advantage of NAS methods [26], [29], [33].

In summary, compared with the compared handcrafted CNN methods for HSI classification, the proposed EB-CNN presents its superiority to them no matter in terms of the overall performance or with respect to the classification map. In competition with the first automatic CNN method (3D-Auto-CNN),

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
OPTIMAL ARCHITECTURE FOUND ON IN

Code	Type	Unit information
13	CU4	in = 200, out = 148, count = 7
22	PU2	pool type = max pooling
21	PU1	pool type = average pooling
22	PU2	pool type = max pooling
22	PU2	pool type = max pooling

TABLE IX
OPTIMAL ARCHITECTURE FOUND ON SA

Code	Type	Cache information
12	CU3	in = 200, out = 128, count = 7
21	PU1	pool type = average pooling
12	CU3	in = 128, out = 256, count = 7
10	CU1	in = 256, out = 256, count = 10
12	CU3	in = 256, out = 256, count = 5
21	PU1	pool type = average pooling
22	PU2	pool type = max pooling
22	PU2	pool type = max pooling
13	CU4	in = 256, out = 64, count = 8

TABLE X
OPTIMAL ARCHITECTURE FOUND ON UP

Code	Type	Unit information
10	CU1	in = 103, out = 96, count = 4
22	PU2	pool type = max pooling
22	PU2	pool type = max pooling
13	CU4	in = 96, out = 231, count = 6
22	PU2	pool type = max pooling

TABLE XI
OPTIMAL ARCHITECTURE FOUND ON HU2013

Code	Type	Unit information
10	CU1	in = 144, out = 144, count = 3
10	CU1	in = 144, out = 177, count = 3
20	PU1	pool type = average pooling

TABLE XII
OPTIMAL ARCHITECTURE FOUND ON HU2018

Code	Type	Unit information
10	CU1	in = 50, out = 50, count =7
20	PU1	pool type = average pooling
10	CU1	in = 50, out = 121, count = 8
21	PU2	pool type = max pooling

EB-CNN achieves slightly better performance. Though both 3D-Auto-CNN and EB-CNN could automatically find optimal architectures for CNN, they search for the optimal architecture in very different ways. Specifically, 3D-Auto-CNN keeps the overall architecture fixed and only optimizes the operation types of the components, while EB-CNN simultaneously optimizes the width and the depth of the architecture and the components in each layer. Therefore, EB-CNN requires much less expert knowledge and experience on the architecture design of CNN than 3D-Auto-CNN. From this perspective, the usability and applicability of EB-CNN are better and thus can be better accessible to those, who have great demand on HSI classification but with very limited knowledge on CNN.

D. Investigation of the Proposed EB-CNN

This section takes deep investigation on the proposed EB-CNN to dig out the found optimal architectures on the five datasets, the evolutionary trajectory in seeking the optimal architectures and the influence of newly introduced swapping mutation operation.

1) Found Optimal CNN Architectures: Tables VIII–XII show the optimal CNN architectures found by EB-CNN on the IN, SA, UP, HU2013, and HU2018 datasets in one random run, respectively. In particular, the learning curves of these optimal architectures regarding the accuracy and loss on the training and validation sets of the five datasets are shown in Fig. S6 in the supplementary material. Furthermore, Table SIII in the supplementary material shows the complexity comparisons between the optimal CNNs found by EB-CNN and the compared CNNs with respect to the floating point of operations (FLOPs), and the number of parameters on the five datasets, respectively.

On the IN dataset, as shown in Table VIII, one optimal CNN architecture found by the proposed EB-CNN contains seven 5×5 residual dilated separable convolution blocks, one 3×3 max PB, one 3×3 average PB, and two 3×3 max PBs in order. The total number of layers is 11 $(7 + 1 + 1 + 1 + 1)$. As shown in Fig. S6(a) in the supplementary material, as the training epoch increases, the training accuracy of the optimal CNN architecture nearly keeps increasing to (nearby) 100%, while the validation accuracy fluctuates seriously at the early stage and then gradually stays steady around 100% as well. As shown in Table SIII in the supplementary material, among the six architectures, the optimal architecture found by EB-CNN has the third least FLOPs. This is because there are many channel transformations in the found architecture, as shown in Table VIII.

On the SA dataset, as shown in Table IX, one optimal CNN architecture found by EB-CNN is composed in sequential of seven 3×3 residual dilated separable convolution blocks, one 3×3 average PB, seven 3×3 residual dilated separable convolution blocks, ten 3×3 residual separable convolution blocks, five 3×3 residual dilated separable convolution blocks, one 3×3 average PB, two continuous 3×3 max PBs, and eight 5×5 residual dilated separable convolution blocks. The total number of layers is 41 $(7+1+7+10+5+1+1+1+8)$. As shown in Fig. S6 (b) in the supplementary material, the training accuracy of the optimal CNN architecture also nearly keeps increasing until nearby 100%, while the validation accuracy fluctuates at first and then converges around 100% as well. As shown in Table SIII in the supplementary material, compared with the 5 existing CNN-based methods, the optimal architecture found by EB-CNN has the most FLOPs. This is because there are many channels in the used CUs in the found optimal architecture, as shown in Table IX.

On the UP dataset, as shown in Table VIII, one optimal CNN architecture found by the proposed EB-CNN is comprised sequentially by four 3×3 residual separable convolution blocks, one 3×3 max PB, six 5×5 residual dilated separable convolution blocks, and one 3×3 max PB. The total number of layers is 13 $(4+1+1+6+1)$. As shown

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

in Fig. S6(c) in the supplementary material, as the training process goes, the training accuracy of the optimal CNN architecture nearly keeps increasing to almost 100%, while the validation accuracy fluctuates at the early stage and then stays steady around 100%. As shown in Table SIII in the supplementary material, among the six CNN methods, the optimal architecture found by EB-CNN has the second least FLOPs.

On the HU2013 dataset [61], as shown in Table VIII, one optimal CNN architecture found by the proposed EB-CNN is comprised sequentially by three 3×3 residual separable convolution blocks, another three 3×3 residual separable convolution blocks, and one 3×3 average PB. The total number of layers is 7 $(3 + 3 + 1)$. As shown in Fig. S6(d) in the supplementary material, as the training process goes, the training accuracy of the optimal CNN architecture nearly keeps increasing to almost 100%, while the validation accuracy fluctuates around 85%. As shown in Table SIII in the supplementary material, among the six architectures, the one found by EB-CNN has the third least FLOPs.

On the HU2018 dataset [62], as shown in Table X, one optimal CNN architecture found by the proposed EB-CNN consists of seven 3×3 residual separable convolution blocks, one 3×3 average PB, eight another 3×3 residual separable convolution blocks, and one 3×3 max PB. The total number of layers is 17 $(7 + 1 + 8 + 1)$. As shown in Fig. S6(e), in the supplementary material, as the training process goes, the training accuracy of the optimal CNN architecture still nearly keeps increasing to almost 100%, while the validation accuracy fluctuates around 65%. As shown in Table SIII in the supplementary material, among the six architectures, the one found by EB-CNN has the second least FLOPs.

In summary, it is not difficult to find that on different datasets, the optimal architectures are very different with respect to not only the depth and width of the architecture, but also the combination of the predefined operations. This mainly benefits from the proposed flexible encoding scheme and the evolution operators. Specifically, armed with the two techniques, EB-CNN not only optimizes the depth (in two ways: the length of the chromosome and the number of CBs in CUs) and the width (by optimizing the number of the output channels in CUs) of the architecture simultaneously, but also optimizes the combination of operations in the components of chromosomes. From this perspective, EB-CNN can be considered as a completely automatic NAS method.

2) Evolutionary Trajectory: Taking the IN dataset as an example, this section conducts experiments to investigate the evolutionary search process of GA when seeking the optimal architecture. In the experiment, both the number of chromosomes and the number of generations are set to 20. Fig. 11 presents the change of the global best fitness, the average fitness, and the worst fitness of the whole population in GA as the evolution goes. It deserves attention that the fitness of a chromosome actually is the overall classification accuracy (OA) of the constructed CNN on the validation set.

From this figure, the following findings are obtained. 1) As the iteration continues, the global best fitness first gradually increases and then keeps unchanged after nine generations,

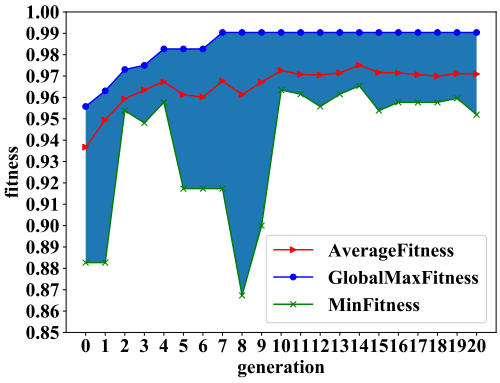


Fig. 11. Evolutionary trajectory of the proposed EB-CNN in IN.

as shown in the blue line in Fig. 11. This indicates that after nine generations, the population evolved by GA could not find a better CNN architecture. 2) The worst fitness in each generation fiercely fluctuates before ten generations and then fluctuates mildly after ten generations. This phenomenon is normal because during the generation of offspring, it is likely to generate poor individuals due to the randomness of the mutation operations. However, in the late stages, the population might converge to promising areas and the CNN architectures encoded in the chromosomes are very similar. Therefore, the difference among offspring becomes much smaller in the late stages than that in the early stages. 3) As for the average fitness of the whole population, it gradually becomes better and better as the evolution iterates. This indicates that the population gradually approaches more and more promising areas. However, due to the randomness of the mutation operations, all chromosomes in the population are less likely to converge to one point, namely the global best chromosome. This is in turn beneficial for the diversity enhancement of the population, which makes great contributions to finding a more promising area and escaping from local areas.

Based on the above observation, it is enough to set the number of generations to 10 for EB-CNN to find a promising CNN architecture. Since the fitness evaluation of chromosomes involves training and validating the associated CNNs on the training and validation datasets, such a number is also suitable to compromise the search performance and speed of EB-CNN.

3) Impact of the Swapping Operator: As aforementioned in Section III-D, to accelerate the evolution of GA to find an optimal CNN architecture, this article devises a swapping mutation operation, which can be actually considered as a local search method. Therefore, to investigate the impact of this operation, this section takes the IN dataset as an example to conduct experiments on EB-CNN. Fig. 12 presents the performance comparison between the EB-CNN with the swapping mutation operation and the one without the swapping mutation operation with respect to the change of the global best fitness as the evolution goes. In this figure, if EB-CNN selects the swapping mutation to generate offspring in one generation, a tag named "swap" is added along with the global fitness change curve, as shown in the red line in Fig. 12.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

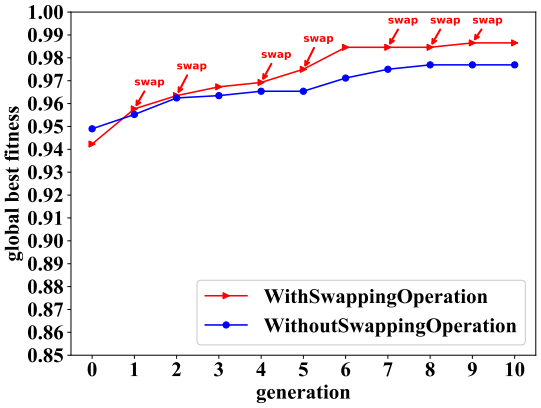


Fig. 12. Comparison of EB-CNN with and without the swapping operator on dataset IN.

From this figure, it is not difficult to find that EB-CNN with the swapping mutation achieves much better performance than the one without the swapping operation. This indicates that the swapping mutation plays an important role in helping EB-CNN achieve promising performance. Particularly, as a special kind of local search operation, the swapping mutation exchanges components in the same chromosome to fine-tune the architecture and thus the classification performance is likely to be improved. In addition, as analyzed in Section III-D, the swapping mutation operation could also improve the convergence speed of EB-CNN to promising areas, which can be seen from Fig. 12. This is mainly because it allows GA to use much fewer steps (generations) to transform a chromosome (an architecture) to another chromosome.

V. Conclusion

This article has proposed an evolving block-based convolution neural network (EB-CNN) for HSI classification. Specifically, this article considers the optimization of the CNN architecture as a kind of combinational optimization problem. To solve this problem effectively, this article adopts the GA as the optimizer. Then, this article designs two kinds of blocks, namely the convolution block (CB) and the pooling block (PB), to predefine the search space. Particularly, four different types of CBs and two types of PBs, which are all commonly used in the literature, are devised to form the general search space. Subsequently, a flexible encoding scheme of the chromosome is specifically put forward to allow different chromosomes to have different lengths. Besides, existing crossover strategies and mutation strategies are adapted to this encoding schemes to generate offspring. To further improve the evolution effectiveness and efficiency, a swapping mutation is devised as a special local search to fine-tune the CNN architecture.

In contrast to existing automatic CNNs for HSI classification, which usually keep the overall architecture fixed and only optimize the operation types in the components of the architecture, the proposed EB-CNN not only simultaneously optimizes the width and the depth of the architecture, but also optimizes the combinations of operation types. Such a unique merit benefits from the proposed encoding scheme for GA and results in further reduction of dependence on expert knowledge and experience on CNN and architecture design. From this perspective, EB-CNN, as far as we know, is the first completely automatic NAS algorithm for HSI classification. Therefore, EB-CNN preserves good usability and applicability, and thus can be accessible to those, who have great demand on HSI classification but with very limited knowledge on CNN.

Extensive experiments on five well-known HSI classification datasets have demonstrated the effectiveness of the proposed EB-CNN as compared with 9 state-of-the-art HSI classification methods. Deep investigation on the proposed EB-CNN has also been taken to observe the found optimal architectures and the influence of the newly introduced mutation operator.

In the future, we will attempt to expand the search space of EB-CNN by predefining the attention block, the self-attention block, or even the transformer block, which have been demonstrated to be effective for HSI classification. In this way, the proposed EB-CNN is expected to construct more promising CNN architectures for HSI classification.

References

[1] S. Jingjing, Z. Yuxing, and X. Chaozong, "Forest resources annual dynamic monitoring based on high resolution remote sensing data," in *Proc. IEEE Geosci. Remote Sens. Symp.*, Jul. 2014, pp. 749–752.

[2] T. J. Cudahy, L. B. Whitbourn, P. M. Connor, P. Mason, and R. N. Phillips, "Mapping surface mineralogy and scattering behavior using backscattered reflectance from a hyperspectral midinfrared airborne CO_2 laser system (MIRACO$_2$LAS)," *IEEE Trans. Geosci. Remote Sens.*, vol. 37, no. 4, pp. 2019–2034, Jul. 1999.

[3] Q. Shen *et al.*, "A CIE color purity algorithm to detect black and odorous water in urban rivers using high-resolution multispectral remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6577–6590, Sep. 2019.

[4] L. Zhang, Y. Zhong, B. Huang, J. Gong, and P. Li, "Dimensionality reduction based on clonal selection for hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 45, no. 12, pp. 4172–4186, Dec. 2007.

[5] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Spectral-spatial classification of hyperspectral data using loopy belief propagation and active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 2, pp. 844–856, Feb. 2013.

[6] B. Rasti *et al.*, "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[7] J. Wang *et al.*, "NAS-guided lightweight multiscale attention fusion network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8754–8767, Oct. 2021.

[8] M. Ahmad *et al.*, "Hyperspectral image classification–traditional to deep models: A survey for future prospects," 2021, *arXiv:2101.06116*.

[9] S. Li, W. Song, L. Fang, Y. Chen, P. Ghamisi, and J. A. Benediktsson, "Deep learning for hyperspectral image classification: An overview," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 6690–6709, Sep. 2019.

[10] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[11] Y. Chen, N. M. Nasrabadi, and T. D. Tran, "Classification for hyperspectral imagery based on sparse representation," in *Proc. 2nd Workshop Hyperspectral Image Signal Process., Evol. Remote Sens.*, Jun. 2010, pp. 1–4.

[12] J. Li, J. Bioucas-Dias, and A. Plaza, "Spectral–spatial hyperspectral image segmentation using subspace multinomial logistic regression and Markov random fields," *IEEE Trans. Geosci. Remote Sens.*, vol. 50, no. 3, pp. 809–823, Mar. 2012.

[13] M. Imani and H. Ghassemian, "An overview on spectral and spatial information fusion for hyperspectral image classification: Current trends and challenges," *Inf. Fusion*, vol. 59, pp. 59–83, Jul. 2020.

[14] R. Li, S. Zheng, C. Duan, Y. Yang, and X. Wang, "Classification of hyperspectral image based on double-branch dual-attention mechanism network," *Remote Sens.*, vol. 12, no. 3, pp. 582–607, 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.

[15] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral-spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.

[16] L. Huang, C. Chen, W. Li, and Q. Du, "Remote sensing image scene classification using multi-scale completed local binary patterns and Fisher vectors," *Remote Sens.*, vol. 8, no. 6, pp. 483–500, 2016.

[17] D. G. Lowe, "Object recognition from local scale-invariant features," in *Proc. 7th IEEE Int. Conf. Comput. Vis.*, vol. 2, Sep. 1999, pp. 1150–1157.

[18] S. Valero, P. Salembier, and J. Chanussot, "Hyperspectral image representation and processing with binary partition trees," *IEEE Trans. Image Process.*, vol. 22, no. 4, pp. 1430–1443, Apr. 2013.

[19] M. Pal, "Ensemble of support vector machines for land cover classification," *Int. J. Remote Sens.*, vol. 29, no. 10, pp. 3043–3049, May 2008.

[20] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[21] J. Peng, Y. Zhou, and C. L. P. Chen, "Region-kernel-based support vector machines for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 9, pp. 4810–4824, Sep. 2015.

[22] J. Li, X. Zhao, Y. Li, Q. Du, B. Xi, and J. Hu, "Classification of hyperspectral imagery using a new fully convolutional neural network," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 2, pp. 292–296, Feb. 2018.

[23] X. Yang, Y. Ye, X. Li, R. Y. K. Lau, X. Zhang, and X. Huang, "Hyperspectral image classification with deep learning models," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 9, pp. 5408–5423, Sep. 2018.

[24] Z. Lu, B. Xu, L. Sun, T. Zhan, and S. Tang, "3-D channel and spatial attention based multiscale spatial–spectral residual network for hyperspectral image classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4311–4324, 2020.

[25] Q. Yao *et al.*, "Taking human out of learning applications: A survey on automated machine learning," 2018, *arXiv:1810.13306*.

[26] H. X. Liu, K. Simonyan, and Y. M. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–13.

[27] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 55, pp. 1–21, 2019.

[28] E. Real *et al.*, "Large-scale evolution of image classifiers," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2902–2911.

[29] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Completely automated CNN architecture design based on blocks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 4, pp. 1242–1254, Apr. 2020.

[30] Z. Yang *et al.*, "CARS: Continuous evolution for efficient neural architecture search," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1826–1835.

[31] Y. Sun, B. Xue, M. Zhang, and G. G. Yen, "Evolving deep convolutional neural networks for image classification," *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 394–407, Apr. 2020.

[32] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically designing CNN architectures using the genetic algorithm for image classification," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3840–3854, Sep. 2020.

[33] L. Xie and A. Yuille, "Genetic CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2017, pp. 1388–1397.

[34] Z. Chen and B. Li, "Efficient evolution for neural architecture search," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–7.

[35] H. Zhang, Y. Jin, R. Cheng, and K. Hao, "Efficient evolutionary search of attention convolutional networks via sampled training and node inheritance," *IEEE Trans. Evol. Comput.*, vol. 25, no. 2, pp. 371–385, Apr. 2021.

[36] Y. Chen, K. Zhu, L. Zhu, X. He, P. Ghamisi, and J. A. Benediktsson, "Automatic design of convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7048–7066, Apr. 2019.

[37] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral-spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, May 2015.

[38] Y. Li, H. K. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, pp. 67–87, 2017.

[39] Y. Chen, H. Jiang, C. Li, X. Jia, and P. Ghamisi, "Deep feature extraction and classification of hyperspectral images based on convolutional neural networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6232–6251, Oct. 2016.

[40] W. Li, G. Wu, F. Zhang, and Q. Du, "Hyperspectral image classification using deep pixel-pair features," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 844–853, Feb. 2017.

[41] M. Zhang, W. Li, and Q. Du, "Diverse region-based CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 27, no. 6, pp. 2623–2634, Jun. 2018.

[42] H. Lee and H. Kwon, "Going deeper with contextual CNN for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 26, no. 10, pp. 4843–4855, Oct. 2017.

[43] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 1–12.

[44] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral–spatial residual network for hyperspectral image classification: A 3-D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[45] M. Zhu, L. Jiao, F. Liu, S. Yang, and J. Wang, "Residual spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 449–462, Jan. 2021.

[46] H.-C. Li, S.-S. Li, W.-S. Hu, J.-H. Feng, W.-W. Sun, and Q. Du, "Recurrent feedback convolutional neural network for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[47] C. Chen, Y. Ma, and G. Ren, "Hyperspectral classification using deep belief networks based on conjugate gradient update and pixel-centric spectral block features," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4060–4069, 2020.

[48] R. Hang, F. Zhou, Q. Liu, and P. Ghamisi, "Classification of hyperspectral images via multitask generative adversarial networks," *IEEE Trans. Geoence Remote Sens.*, vol. 59, no. 2, pp. 1424–1436, Feb. 2020.

[49] R. Lei *et al.*, "A non-local capsule neural network for hyperspectral remote sensing image classification," *Remote Sens. Lett.*, vol. 12, no. 1, pp. 40–49, Jan. 2021.

[50] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 7, pp. 5966–5978, Jul. 2021.

[51] Q. Liu, L. Xiao, J. Yang, and Z. Wei, "CNN-enhanced graph convolutional network with pixel- and superpixel-level feature fusion for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 10, pp. 8657–8671, Oct. 2021.

[52] J. He, L. Zhao, H. Yang, M. Zhang, and W. Li, "HSI-BERT: Hyperspectral image classification using the bidirectional encoder representation from transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 165–178, Jan. 2020.

[53] D. Hong *et al.*, "SpectralFormer: Rethinking hyperspectral image classification with transformers," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–15, 2022.

[54] G. Huang, Z. Liu, L. V. D. Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jul. 2017, pp. 2261–2269.

[55] G. X. Zhang, Y. J. Gu, L. Z. Hu, and W. D. Jin, "A novel genetic algorithm and its application to digital filter design," in *Proc. IEEE Int. Conf. Intell. Transp. Syst.*, vol. 2, Oct. 2003, pp. 1600–1605.

[56] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[57] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 1–8.

[58] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," 2015, *arXiv:1511.07122*.

[59] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, 2015, pp. 1–15.

[60] J. A. Vasconcelos, J. A. Ramirez, R. H. C. Takahashi, and R. R. Saldanha, "Improvements in genetic algorithms," *IEEE Trans. Magn.*, vol. 37, no. 5, pp. 3414–3417, Sep. 2001.

[61] C. Debes *et al.*, "Hyperspectral and LiDAR data fusion: Outcome of the 2013 GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 7, no. 6, pp. 2405–2418, Jun. 2014.

[62] Y. Xu *et al.*, "Advanced multi-sensor optical remote sensing for urban land use and land cover classification: Outcome of the 2018 IEEE GRSS data fusion contest," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 6, pp. 1709–1724, Jun. 2019.

[63] G. John, "Evolutionary algorithms in theory and practice: Evolution strategies, evolutionary programming, genetic algorithms," *Kybernetes*, vol. 27, no. 8, pp. 979–980, 1998.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.



Zhenyu Lu (Member, IEEE) received the B.Sc. degree in electricity and the M.Sc. degree in information and communication from the Nanjing Institute of Meteorology, Nanjing, China, in 1999 and 2002, respectively, and the Ph.D. degree in optics engineering from Nanjing University of Science and Technology, Nanjing, in 2008.

He was a Research Associate with the Department of Mathematics and Statistics, University of Strathclyde, Glasgow, U.K., from 2012 to 2013. He is a Professor with the School of Artificial Intelligence (AI), Nanjing University of Information Science and Technology. He has published 20 international journal articles. His research interests include neural networks, stochastic control, and artificial intelligence.



Shaoyang Liang is pursuing the master's degree in information and communication engineering with the School of Electronics and Information Engineering, Nanjing University of Information Science and Technology, Nanjing, China.

His research interests include image classification, neural network search, and deep learning.



Qiang Yang (Member, IEEE) received the M.S. and Ph.D. degrees from Sun Yat-sen University, Guangzhou, China, in 2014 and 2019, respectively.

He is a Professor with the School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing, China. His research interests include evolutionary computation algorithms and their applications on real-world problems. So far, he specifically works on large-scale optimization algorithms, multimodal optimization algorithms, distributed evolutionary algorithms, and their applications on real-world problems, such as intelligent transportation, logistics scheduling optimization, and smart weather.



Bo Du (Senior Member, IEEE) received the Ph.D. degree in photogrammetry and remote sensing from the State Key Laboratory of Information Engineering in Surveying, Mapping and Remote Sensing, Wuhan University, Wuhan, China, in 2010.

He is a Professor with the School of Computer Science, Wuhan University. He has over 60 research articles published in the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING (TGRS), the IEEE TRANSACTIONS ON IMAGE PROCESSING (TIP), the IEEE JOURNAL OF SELECTED TOPICS IN EARTH OBSERVATIONS AND APPLIED REMOTE SENSING (JSTARS), and the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS (GRSL). Thirteen of them are ESI hot articles or highly cited articles. His major research interests include pattern recognition, hyperspectral image processing, machine learning, and signal processing.

Dr. Du was a recipient of the Distinguished Paper Award from IJCAI 2018, the Best Paper Award of the IEEE Whispers 2018, the Champion Award of the IEEE Data Fusion Contest 2018, the Best Reviewer Award from the IEEE GRSS for his service to the IEEE JOURNAL OF SELECTED TOPICS IN EARTH OBSERVATIONS AND APPLIED REMOTE SENSING (JSTARS) in 2011, and the ACM rising star awards for his academic progress in 2015. He was the Session Chair of the International Geoscience and Remote Sensing Symposium (IGARSS) 2018/2016 and the 4th IEEE GRSS Workshop on Hyperspectral Image and Signal Processing: Evolution in Remote Sensing (WHISPERS). He also serves as a Reviewer for 20 Science Citation Index (SCI) magazines, including IEEE TGRS, TIP, JSTARS, and GRSL.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 01:26:53 UTC from IEEE Xplore. Restrictions apply.